

Melanoma DIABLO Study

DIABLO Melanoma Multi-Omics Study

This workflow is based on <https://mixomics.org/mixdiablo/diablo-tcga-case-study/>. It integrates RNA-seq, LFQ intensities and SERS data using DIABLO. The aim is to identify a correlated multi-omics signature that separates 1G and DLN. The transcriptomics part uses only the 2D cell samples. The proteomics part uses the MaxQuant `proteinGroups` file where technical replicates were already merged. In total six groups will be retained 1G_A/_B/_C and DLN_A/_B/_C.

This script provides three approaches for reducing the SERS feature space: sparse group lasso (SGL), a genetic algorithm (GA), and correlation-based variable window selection (WS). The method is selected with the `sers_selection_method` parameter in the YAML header.

Setup

Configure global R Markdown settings, standardize plot parameters, define the project root directory, and ensure reproducibility.

Load libraries

```
library(mixOmics)
library(DESeq2)
library(edgeR)
library(dplyr)
library(igraph)
library(org.Hs.eg.db)
library(AnnotationDbi)
library(BiocParallel)
library(msgl)
library(GA)
library(parallel)
library(doParallel)

if (!requireNamespace("ComplexHeatmap", quietly = TRUE)) {
  stop(
    "Package 'ComplexHeatmap' is required for the labelled DIABLO heatmap. ",
    "Install it with BiocManager::install('ComplexHeatmap')."
  )
}

if (!requireNamespace("circlize", quietly = TRUE)) {
  stop(
    "Package 'circlize' is required for the labelled DIABLO heatmap. ",

```

```

    "Install it with install.packages('circlize')."
  )
}

set.seed(123)

sers_selection_method <- toupper(params$sers_selection_method)
ws_correlation_threshold <- as.numeric(params$ws_correlation_threshold)

if (!sers_selection_method %in% c("SGL", "GA", "WS")) {
  stop("sers_selection_method must be 'SGL', 'GA', or 'WS'.")
}

if (
  length(ws_correlation_threshold) != 1L ||
  !is.finite(ws_correlation_threshold) ||
  ws_correlation_threshold < -1 ||
  ws_correlation_threshold > 1
) {
  stop("ws_correlation_threshold must be a number between -1 and 1.")
}

cat("SERS region-selection method:", sers_selection_method, "\n")

## SERS region-selection method: SGL

if (sers_selection_method == "WS") {
  cat("WS correlation threshold:", ws_correlation_threshold, "\n")
}

```

Define input directories and file paths

```

project_root <- getwd()

rna_dir <- file.path(project_root, "Transcriptome")
prot_dir <- file.path(project_root, "Proteome")
sers_dir <- file.path(project_root, "SERS")

sers_method_tag <- toupper(sers_selection_method)

results_dir <- file.path(
  project_root,
  paste0("DIABLO_", sers_method_tag, "_results")
)

figure_dir <- file.path(
  results_dir,
  paste0(sers_method_tag, "_DIABLO_figures")
)

dir.create(results_dir, recursive = TRUE, showWarnings = FALSE)

```

```
dir.create(figure_dir, recursive = TRUE, showWarnings = FALSE)
```

```
result_file <- function(filename) {
  file.path(
    results_dir,
    paste0(sers_method_tag, "_", filename)
  )
}
```

```
cat("Method-specific results directory:", results_dir, "\n")
```

```
## Method-specific results directory: /home/fhwn.ac.at/116914/DIABLO_Melanoma_Pauser/DIABLO_SGL_results
```

```
prot_file <- file.path(
  prot_dir,
  "proteinGroups_results_run1_mqpar_2025__FH_250515_Mayr_extracts_mcm_new_merged.txt.txt"
)
```

```
sers_file <- file.path(
  sers_dir,
  "melamonma_g_dln_sers_rerun_annamasterthesis_preprocessed.csv"
)
```

```
stopifnot(
  dir.exists(rna_dir),
  dir.exists(prot_dir),
  dir.exists(sers_dir),
  file.exists(prot_file),
  file.exists(sers_file)
)
```

Helper functions for sample-name harmonisation and data aggregation

```
clean_sample_names <- function(x) {
  x <- basename(x)
  x <- gsub("[.]bam$|[.]sorted$|[.]rmdup$|[.]counts$|[.]txt$|[.]tsv$|[.]csv$", "", x, ignore.case = TRUE)
  x <- gsub("[.]featurecount$|[.]featurecounts$|_featurecount$|_featurecounts$|_counts$", "", x, ignore.case = TRUE)
  x <- gsub("LFQ[.]intensity[.]", "", x, ignore.case = TRUE)
  x <- gsub("Intensity[.]", "", x, ignore.case = TRUE)
  x <- gsub("Reporter[.]intensity[.]", "", x, ignore.case = TRUE)
  x <- gsub("[^A-Za-z0-9]+", "_", x)
  x <- gsub("^_|_$", "", x)
  x
}
```

```
make_unique <- function(x) {
  x <- ifelse(is.na(x) | x == "", "feature", x)
  make.unique(x)
}
```

```
get_group <- function(x) {
```

```

  sapply(strsplit(as.character(x), "_"), function(z) z[1])
}

get_biorep <- function(x) {
  sapply(strsplit(as.character(x), "_"), function(z) ifelse(length(z) >= 2, z[2], NA_character_))
}

collapse_columns_by_name_sum <- function(mat) {
  ids <- colnames(mat)
  ids_unique <- unique(ids)

  collapsed <- sapply(ids_unique, function(id) {
    cols <- which(ids == id)
    rowSums(mat[, cols, drop = FALSE], na.rm = TRUE)
  })

  collapsed <- as.matrix(collapsed)
  colnames(collapsed) <- ids_unique
  collapsed
}

harmonise_rna_sample_names <- function(x) {
  x <- basename(x)
  x <- gsub("[.]featurecount$|[/]featurecounts$", "", x, ignore.case = TRUE)
  x <- gsub("_STAR.*$", "", x, ignore.case = TRUE)
  x <- gsub("-", "_", x)
  x <- gsub("[^A-Za-z0-9]+", "_", x)
  x <- gsub("^_|_$", "", x)

  rep_map <- c("1" = "A", "2" = "B", "3" = "C")
  out <- rep(NA_character_, length(x))

  for (i in seq_along(x)) {
    parts <- strsplit(x[i], "_")[[1]]

    if (length(parts) == 2 && parts[1] == "1G" && parts[2] %in% names(rep_map)) {
      out[i] <- paste0("1G_", rep_map[parts[2]])
    }

    if (length(parts) == 3 && parts[1] == "DLN" && parts[2] == "2D" && parts[3] %in% names(rep_map)) {
      out[i] <- paste0("DLN_", rep_map[parts[3]])
    }
  }

  out
}

harmonise_sers_sample_names <- function(x) {
  x <- basename(x)
  group <- ifelse(grepl("_1G_", x), "1G", ifelse(grepl("_DLN_", x), "DLN", NA_character_))
  m <- sub(".*_M([123])[*].*$", "\\1", x)
  rep_map <- c("1" = "A", "2" = "B", "3" = "C")
  rep <- rep_map[m]

```

```

    ifelse(!is.na(group) & !is.na(rep), paste0(group, "_", rep), NA_character_)
  }

harmonise_prot_sample_names <- function(x) {
  x <- clean_sample_names(x)
  x <- gsub("^LFQ_intensity_", "", x, ignore.case = TRUE)
  x <- gsub("^LFQ_intensity", "", x, ignore.case = TRUE)
  x <- gsub("^Intensity_", "", x, ignore.case = TRUE)
  x <- gsub("^Reporter_intensity_", "", x, ignore.case = TRUE)
  x <- gsub("_2D_", "_", x, ignore.case = TRUE)
  x <- gsub("DLN_2D_", "DLN_", x, ignore.case = TRUE)
  x <- gsub("[^A-Za-z0-9]+", "_", x)
  x <- gsub("^_|_$", "", x)

  rep_map <- c("1" = "A", "2" = "B", "3" = "C")
  out <- rep(NA_character_, length(x))

  for (i in seq_along(x)) {
    parts <- strsplit(x[i], "_")[[1]]

    if (length(parts) >= 2 && parts[1] == "1G") {
      rep_token <- parts[2]
      if (rep_token %in% names(rep_map)) rep_token <- rep_map[rep_token]
      if (rep_token %in% c("A", "B", "C")) out[i] <- paste0("1G_", rep_token)
    }

    if (length(parts) >= 2 && parts[1] == "DLN") {
      rep_token <- parts[length(parts)]
      if (rep_token %in% names(rep_map)) rep_token <- rep_map[rep_token]
      if (rep_token %in% c("A", "B", "C")) out[i] <- paste0("DLN_", rep_token)
    }
  }
  out
}

```

Define and verify the RNA-seq input files

```

rna_files <- file.path(
  rna_dir,
  c(
    "1G-1_STAR_WGF_sorted.rmdup.featurecount",
    "1G-2_STAR_WGF_sorted.rmdup.featurecount",
    "1G-3_STAR_WGF_sorted.rmdup.featurecount",
    "DLN-2D-1_STAR_WGF_sorted.rmdup.featurecount",
    "DLN-2D-2_STAR_WGF_sorted.rmdup.featurecount",
    "DLN-2D-3_STAR_WGF_sorted.rmdup.featurecount"
  )
)

missing_rna_files <- rna_files[!file.exists(rna_files)]

```

```

if (length(missing_rna_files) > 0) {
  stop(
    "Missing RNA-seq files: ",
    paste(missing_rna_files, collapse = ", ")
  )
}

stopifnot(length(rna_files) == 6)

```

Read, merge, and format the RNA-seq count data

```

read_featurecounts <- function(f) {
  x <- read.delim(f, comment.char = "#", check.names = FALSE)
  if (!"Geneid" %in% colnames(x)) stop(paste("No Geneid column in", f))

  annotation_cols <- intersect(c("Geneid", "Chr", "Start", "End", "Strand", "Length"), colnames(x))
  count_cols <- setdiff(colnames(x), annotation_cols)
  count_cols <- count_cols[sapply(x[count_cols], is.numeric)]

  out <- x[, c("Geneid", count_cols), drop = FALSE]
  colnames(out)[-1] = harmonise_rna_sample_names(f)
  out
}

rna_list <- lapply(rna_files, read_featurecounts)

if (length(rna_list) == 1) {
  rna_counts <- rna_list[[1]]
} else {
  rna_counts <- Reduce(function(a, b) full_join(a, b, by = "Geneid"), rna_list)
}

rna_counts <- as.data.frame(rna_counts)
rna_gene <- rna_counts$Geneid
rna_counts <- rna_counts[, -1, drop = FALSE]
rownames(rna_counts) = make_unique(rna_gene)

rna_counts[is.na(rna_counts)] = 0
rna_counts <- round(as.matrix(rna_counts))
storage.mode(rna_counts) = "integer"

rna_counts <- collapse_columns_by_name_sum(rna_counts)
storage.mode(rna_counts) = "integer"

expected_samples <- c("1G_A", "1G_B", "1G_C", "DLN_A", "DLN_B", "DLN_C")
rna_counts <- rna_counts[, expected_samples, drop = FALSE]

cat("RNA samples retained:\n")

```

```
## RNA samples retained:
```

```
print(colnames(rna_counts))
```

```
## [1] "1G_A" "1G_B" "1G_C" "DLN_A" "DLN_B" "DLN_C"
```

```
dim(rna_counts)
```

```
## [1] 63677      6
```

```
rna_counts[1:5, 1:min(5, ncol(rna_counts))]
```

```
##           1G_A 1G_B 1G_C DLN_A DLN_B
## ENSG00000223972    19   10   16    12    8
## ENSG00000227232  1095  851  906   673   879
## ENSG00000243485    0    0    0    0    0
## ENSG00000237613    0    0    0    0    0
## ENSG00000268020    0    0    0    0    0
```

Load and inspect the proteomics data

```
if (!file.exists(prot_file)) {
  stop(paste("Proteomics file does not exist:", prot_file))
}
```

```
prot_raw <- read.delim(prot_file, check.names = FALSE)
dim(prot_raw)
```

```
## [1] 3651    78
```

```
head(prot_raw[, 1:min(6, ncol(prot_raw))])
```

```
##
## 1           Q00341;A0A024R4E5;H0Y394;H7C0A4;C9JIZ1;C9J5E5;H7C2D1;H7BZC3;C9JZI8
## 2
## 3
## 4
## 5
## 6 P46379;A0A7I2V508;A0A0G2JK23;A0A024RCR6;A0A7POMQS5;A0A0G2JL47;A0A0G2JJM1;X6REW1;F6S6P2;F6UR09;F6U3
##           Majority protein IDs
## 1           Q00341;A0A024R4E5;H0Y394
## 2           A0A024R571;Q9H4M9;C9JC03;C9J2Z4
## 3           A0A024R7W5;A0A087X0Q1;A0A087WY31;Q9BYJ9;Q7Z739
## 4           A0A024RA52;P25787;A0A7I2YQP7;C9JCK5;A0A7I2V2H3
## 5           A0A024RBT2;Q9NR28;H7BZQ7;H7BZK7;A0A2U3TZH2
## 6 P46379;A0A7I2V508;A0A0G2JK23;A0A024RCR6;A0A7POMQS5;A0A0G2JL47
##           Peptide counts (all)
## 1 28;28;20;12;9;9;7;6;6;5;5;4;2;1;1;1;1;1;1
## 2          16;16;12;8;5;5;5
## 3          1;1;1;1;1
```

```

## 4          9;9;5;5;5;2
## 5          2;2;1;1;1
## 6      7;7;7;7;7;3;3;3;1;1;1;1;1;1;1;1;1;1;1;1;1
##      Peptide counts (razor+unique)
## 1 28;28;20;12;9;9;7;6;6;5;5;4;2;2;1;1;1;1;1;1;1
## 2          15;15;11;8;4;4;5
## 3          1;1;1;1;1
## 4          9;9;5;5;5;2
## 5          2;2;1;1;1
## 6      7;7;7;7;7;3;3;3;1;1;1;1;1;1;1;1;1;1;1;1;1
##      Peptide counts (unique)
## 1 28;28;20;12;9;9;7;6;6;5;5;4;2;2;1;1;1;1;1;1;1
## 2          9;9;5;3;3;3;1
## 3          1;1;1;1;1
## 4          9;9;5;5;5;2
## 5          2;2;1;1;1
## 6      7;7;7;7;7;3;3;3;1;1;1;1;1;1;1;1;1;1;1;1;1
##
##                                     Protein names
## 1                                     Vigilin
## 2                                     EH domain-containing protein 1
## 3 YTH domain-containing family protein 1;YTH domain-containing family protein 3
## 4      Proteasome subunit alpha type;Proteasome subunit alpha type-2
## 5                                     Diablo homolog, mitochondrial
## 6      Large proline-rich protein BAG6

```

Remove MaxQuant artefact identifications

```

maxquant_flag_columns <- c(
  "Reverse",
  "Potential contaminant",
  "Only identified by site"
)

available_flag_columns <- intersect(
  maxquant_flag_columns,
  colnames(prot_raw)
)

artefact_rows <- rep(FALSE, nrow(prot_raw))

for (flag_column in available_flag_columns) {
  artefact_rows <- artefact_rows |
    trimws(as.character(prot_raw[[flag_column]])) == "+"
}

maxquant_id_columns <- intersect(
  c("Protein IDs", "Majority protein IDs"),
  colnames(prot_raw)
)

if (length(maxquant_id_columns) > 0) {

```

```

artefact_id_rows <- Reduce(
  `|`,
  lapply(
    maxquant_id_columns,
    function(id_column) {
      grepl(
        "(^|;) *(CON_|REV_)",
        as.character(prot_raw[[id_column]]),
        ignore.case = TRUE
      )
    }
  )
)

artefact_rows <- artefact_rows | artefact_id_rows
}

cat("MaxQuant rows before artefact filtering:", nrow(prot_raw), "\n")

```

```
## MaxQuant rows before artefact filtering: 3651
```

```
cat("MaxQuant artefact rows removed:", sum(artefact_rows), "\n")
```

```
## MaxQuant artefact rows removed: 122
```

```
prot_raw <- prot_raw[!artefact_rows, , drop = FALSE]
```

```
cat("MaxQuant rows retained:", nrow(prot_raw), "\n")
```

```
## MaxQuant rows retained: 3529
```

Format, label, and filter the proteomics data without imputation

```

feature_cols <- grep(
  "gene|protein|uniprot|accession|id|name|description",
  colnames(prot_raw),
  ignore.case = TRUE,
  value = TRUE
)

numeric_cols <- colnames(prot_raw)[
  vapply(prot_raw, is.numeric, logical(1))
]

preferred_feature_cols <- c(
  "Gene names", "Gene.names", "Gene name", "Gene.name",
  "Protein names", "Protein.names",
  "Majority protein IDs", "Majority.protein.IDs",
  "Protein IDs", "Protein.IDs"
)

```

```

)

available_feature_cols <- preferred_feature_cols[
  preferred_feature_cols %in% colnames(prot_raw)
]

if (length(available_feature_cols) > 0) {
  feature_col <- available_feature_cols[1]
} else if (length(feature_cols) > 0) {
  feature_col <- feature_cols[1]
} else {
  feature_col <- colnames(prot_raw)[1]
}

# Identify and harmonize sample columns
sample_cols <- grep(
  "LFQ[.]intensity|LFQ intensity|Intensity|Reporter[.]intensity",
  numeric_cols,
  ignore.case = TRUE,
  value = TRUE
)

if (length(sample_cols) == 0) {
  sample_cols <- numeric_cols
}

sample_names <- harmonise_prot_sample_names(sample_cols)
keep_samples <- !is.na(sample_names)

sample_cols <- sample_cols[keep_samples]
sample_names <- sample_names[keep_samples]

stopifnot(length(sample_cols) >= 6)

# Create the protein-abundance matrix
prot_mat <- as.matrix(prot_raw[, sample_cols, drop = FALSE])
storage.mode(prot_mat) <- "double"

protein_labels <- as.character(prot_raw[[feature_col]])
missing_labels <- is.na(protein_labels) | protein_labels == ""
protein_labels[missing_labels] <- as.character(prot_raw[[1]][missing_labels])

protein_labels <- gsub(".*$", "", protein_labels)
protein_labels <- gsub("\\s+", "_", protein_labels)
protein_labels <- substr(protein_labels, 1, 40)

if (any(grepl("^(CON_|REV_)", protein_labels, ignore.case = TRUE))) {
  stop("Contaminant or reverse-sequence identifiers remain in the proteomics data.")
}

rownames(prot_mat) <- make_unique(protein_labels)
colnames(prot_mat) <- sample_names

```

```

# Retain and order the six biological samples
prot_mat <- prot_mat[
  ,
  expected_samples,
  drop = FALSE
]

# MaxQuant represents missing LFQ intensities as either zero or NA
prot_mat[prot_mat == 0] <- NA_real_

# Exclude proteins with missing values; no imputation is applied
prot_mat <- prot_mat[
  complete.cases(prot_mat),
  ,
  drop = FALSE
]

# Remove proteins without variation across samples
prot_mat <- prot_mat[
  apply(prot_mat, 1, sd) > 0,
  ,
  drop = FALSE
]

cat("Protein-label column:", feature_col, "\n")

```

```
## Protein-label column: Gene names
```

```
cat("Retained samples:\n")
```

```
## Retained samples:
```

```
print(colnames(prot_mat))
```

```
## [1] "1G_A" "1G_B" "1G_C" "DLN_A" "DLN_B" "DLN_C"
```

```
dim(prot_mat)
```

```
## [1] 2470    6
```

```
prot_mat[1:5, 1:min(5, ncol(prot_mat))]
```

```
##           1G_A      1G_B      1G_C      DLN_A      DLN_B
## HDLBP  482530000 557480000 400510000 592700000 526850000
## EHD1   138530000 684650000 964100000 470820000 718170000
## YTHDF3 400940000 458790000 344410000 171760000 293370000
## PSMA2  196810000 286960000 177210000 417190000 268150000
## DIABLO  4690300  3188600  1125100  16256000  8699200
```

Load and format the SERS data.

```
if (!file.exists(sers_file)) {
  stop(paste("SERS file does not exist:", sers_file))
}

sers_raw <- read.csv(
  sers_file,
  check.names = FALSE
)

stopifnot(all(c("label", "file") %in% colnames(sers_raw)))

sers_feature_cols <- setdiff(
  colnames(sers_raw),
  c("label", "file")
)

sers_feature_cols <- sers_feature_cols[
  vapply(sers_raw[sers_feature_cols], is.numeric, logical(1))
]

if (length(sers_feature_cols) < 2) {
  stop("Fewer than two numeric SERS variables were detected.")
}

sers_sample_id <- harmonise_sers_sample_names(sers_raw$file)
keep_sers <- !is.na(sers_sample_id)

sers_x_raw <- as.matrix(
  sers_raw[keep_sers, sers_feature_cols, drop = FALSE]
)

storage.mode(sers_x_raw) <- "double"
sers_sample_id <- sers_sample_id[keep_sers]

cat("Spectra per biological sample:\n")
```

Spectra per biological sample:

```
print(table(sers_sample_id))
```

```
## sers_sample_id
## 1G_A 1G_B 1G_C DLN_A DLN_B DLN_C
## 122 107 46 76 95 76
```

```
sers_classes <- factor(
  ifelse(grepl("^1G_", sers_sample_id), "1G", "DLN"),
  levels = c("1G", "DLN")
)
```

```

stopifnot(
  nrow(sers_x_raw) == length(sers_classes),
  ncol(sers_x_raw) == length(sers_feature_cols),
  !anyNA(sers_x_raw),
  all(is.finite(sers_x_raw)),
  length(levels(droplevels(sers_classes))) == 2
)

cat("\nSERS class distribution:\n")

```

```

##
## SERS class distribution:

```

```

print(table(sers_classes))

```

```

## sers_classes
## 1G DLN
## 275 247

```

```

sers_wavenumbers_raw <- suppressWarnings(
  as.numeric(
    gsub(
      "SERS_X|SERS_|X|cm",
      "",
      colnames(sers_x_raw)
    )
  )
)

if (anyNA(sers_wavenumbers_raw)) {
  invalid_names <- colnames(sers_x_raw)[is.na(sers_wavenumbers_raw)]

  stop(
    paste(
      "Could not extract numeric Raman shifts from:",
      paste(invalid_names, collapse = ", ")
    )
  )
}

sers_order <- order(sers_wavenumbers_raw)

sers_wavenumbers_raw <- sers_wavenumbers_raw[sers_order]
sers_x_raw <- sers_x_raw[, sers_order, drop = FALSE]

colnames(sers_x_raw) <- make.unique(
  paste0(
    "SERS_",
    format(
      sers_wavenumbers_raw,
      trim = TRUE,
      scientific = FALSE
    )
  )
)

```

```

    )
  )
)

# Ten-wavenumber groups preserve local spectral structure for SGL and GA.
sgl_window_width <- 10

sers_grouping <- floor(
  (
    sers_wavenumbers_raw -
      min(sers_wavenumbers_raw)
  ) / sgl_window_width
) + 1L

stopifnot(
  length(sers_grouping) == ncol(sers_x_raw),
  all(sers_grouping >= 1)
)

cat("\nSERS input dimensions:\n")

##
## SERS input dimensions:

print(dim(sers_x_raw))

## [1] 522 2201

cat("Number of fixed-width spectral groups:\n")

## Number of fixed-width spectral groups:

print(length(unique(sers_grouping)))

## [1] 111

cat("Variables per spectral group:\n")

## Variables per spectral group:

print(summary(as.numeric(table(sers_grouping))))

##      Min. 1st Qu.  Median    Mean 3rd Qu.   Max.
##      1.00  20.00   20.00   19.83  20.00   20.00

## Define replicate-aware cross-validation folds.

```

```

sers_biorep <- sub(
  "^.*_",
  "",
  sers_sample_id
)

stopifnot(all(sers_biorep %in% c("A", "B", "C")))

# Hold out the corresponding 1G and DLN replicate together.
sgl_cv_indices <- lapply(
  c("A", "B", "C"),
  function(rep_id) {
    which(sers_biorep == rep_id)
  }
)

names(sgl_cv_indices) <- paste0(
  "Leave_out_",
  c("A", "B", "C")
)

cat("\nSGL cross-validation fold sizes:\n")

```

```

##
## SGL cross-validation fold sizes:

print(vapply(sgl_cv_indices, length, integer(1)))

```

```

## Leave_out_A Leave_out_B Leave_out_C
##           198           202           122

cat("\nClass distribution within validation folds:\n")

```

```

##
## Class distribution within validation folds:

```

```

print(
  lapply(
    sgl_cv_indices,
    function(id) table(sers_classes[id])
  )
)

```

```

## $Leave_out_A
##
## 1G DLN
## 122 76
##
## $Leave_out_B
##
## 1G DLN

```

```
## 107 95
##
## $Leave_out_C
##
## 1G DLN
## 46 76

# Ensure that both classes occur in every validation fold
fold_has_both_classes <- vapply(
  sgl_cv_indices,
  function(idx) {
    length(unique(sers_classes[idx])) == 2
  },
  logical(1)
)

if (!all(fold_has_both_classes)) {
  stop("At least one validation fold does not contain both classes.")
}
```

Fit and evaluate the Sparse Group LASSO models.

```
set.seed(123)

if (sers_selection_method == "SGL") {

  detected_cores <- parallel::detectCores(logical = FALSE)

  if (is.na(detected_cores)) {
    detected_cores <- 1
  }

  n_cores <- max(
    1,
    min(2, detected_cores)
  )

  cl <- parallel::makeCluster(n_cores)
  doParallel::registerDoParallel(cl)

  sgl_cv <- tryCatch(
    {
      msgl::cv(
        x = sers_x_raw,
        classes = sers_classes,
        grouping = sers_grouping,

        # alpha = 0: Group LASSO
        # alpha = 1: conventional LASSO
        # 0 < alpha < 1: Sparse Group LASSO
        alpha = 0.5,

```

```

    # Fit 100 models from lambda.max to
    # 0.1 * lambda.max
    lambda = 0.1,
    d = 100,

    fold = 3,
    cv.indices = sgl_cv_indices,

    standardize = TRUE,
    use_parallel = TRUE
  )
},
finally = {
  parallel::stopCluster(cl)
  foreach::registerDoSEQ()
}
)

print(sgl_cv)

# Determine the best cross-validated model

sgl_error <- sglOptim::Err(sgl_cv)

if (
  length(sgl_error) == 0 ||
  all(!is.finite(sgl_error))
) {
  stop("No valid cross-validation errors were returned by msgl.")
}

minimum_cv_error <- min(
  sgl_error,
  na.rm = TRUE
)

# Prefer the sparsest model when several candidates have the same CV error.
best_candidates <- which(
  is.finite(sgl_error) &
  abs(sgl_error - minimum_cv_error) < sqrt(.Machine$double.eps)
)

cat("Tied CV models:", paste(best_candidates, collapse = ", "), "\n")

cat(
  "Minimum CV error:",
  minimum_cv_error,
  "\n"
)
}

```

```
## Running msgl 3 fold cross validation (dense design matrix)
```

```
##
## Samples: Features: Classes: Groups: Parameters:
##      522      2.202k          2      112      4.404k
##
##
## Call:
## msgl::cv(x = sers_x_raw, classes = sers_classes, grouping = sers_grouping,
##      alpha = 0.5, standardize = TRUE, lambda = 0.1, d = 100, fold = 3,
##      cv.indices = sgl_cv_indices, use_parallel = TRUE)
##
## Models:
##
## Index: Lambda: Features: Parameters: Error:
##      1      1.00      1.333      15.333  0.561
##     20      0.64      3.667     108.667  0.084
##     40      0.40         7       242    0.063
##     60      0.25         9       322    0.042
##     80      0.16     10.333       374    0.036
##    100      0.10     11.667       424    0.038
##
## Best model:
##
## Index: Lambda: Features: Parameters: Error:
##     82      0.15         10     360.667  0.034
##
## Tied CV models: 82, 83, 84
## Minimum CV error: 0.03448276
```

##Extract the selected spectral features and regions

```
set.seed(123)

if (sers_selection_method == "SGL") {

sgl_fit <- msgl::fit(
  x = sers_x_raw,
  classes = sers_classes,
  grouping = sers_grouping,
  alpha = 0.5,
  lambda = 0.1,
  d = 100,
  standardize = TRUE
)

sgl_feature_path <- features(sgl_fit)

if (length(sgl_feature_path) != length(sgl_error)) {
  stop(
    paste(
      "The fitted model path and CV error path have different lengths:",
      length(sgl_feature_path),
      "versus",
      length(sgl_error)
    )
  )
}
```

```

)
}

# Exclude the intercept when comparing tied models.
candidate_feature_counts <- vapply(
  best_candidates,
  function(model_index) {

    selected <- setdiff(
      sgl_feature_path[[model_index]],
      "Intercept"
    )

    sum(selected %in% colnames(sers_x_raw))
  },
  integer(1)
)

candidate_summary <- data.frame(
  model_index = best_candidates,
  cv_error = sgl_error[best_candidates],
  selected_wavenumbers = candidate_feature_counts,
  stringsAsFactors = FALSE
)

cat("Tied-model summary:\n")
print(candidate_summary)

sgl_best_index <- best_candidates[
  which.min(candidate_feature_counts)
]

cat(
  "Selected SGL model:",
  sgl_best_index,
  "\n"
)

cat(
  "CV error:",
  sgl_error[sgl_best_index],
  "\n"
)

sers_selected_features <- setdiff(
  sgl_feature_path[[sgl_best_index]],
  "Intercept"
)

sers_selected_features <- intersect(
  sers_selected_features,
  colnames(sers_x_raw)
)

```

```

sers_active_grouping <- sers_grouping

if (length(sers_selected_features) == 0) {
  stop(
    paste(
      "The selected SGL model contains no SERS variables.",
      "Consider reducing lambda, for example from 0.1 to 0.05."
    )
  )
}

cat(
  "Selected SERS wavenumbers:",
  length(sers_selected_features),
  "\n"
)

print(sers_selected_features)

selected_idx <- match(
  sers_selected_features,
  colnames(sers_x_raw)
)

stopifnot(!anyNA(selected_idx))

sers_sgl_selection <- data.frame(
  feature = sers_selected_features,
  wavenumber = sers_wavenumbers_raw[selected_idx],
  group = sers_grouping[selected_idx],
  stringsAsFactors = FALSE
)

sers_sgl_selection <- sers_sgl_selection[
  order(sers_sgl_selection$wavenumber),
  ,
  drop = FALSE
]

sers_sgl_selection$window_start <- min(sers_wavenumbers_raw) +
  (sers_sgl_selection$group - 1) * sgl_window_width

sers_sgl_selection$window_end <- (
  sers_sgl_selection$window_start +
  sgl_window_width
)

sers_selected_regions <- aggregate(
  wavenumber ~ group + window_start + window_end,
  data = sers_sgl_selection,
  FUN = function(z) {
    paste(
      round(sort(z), 2),

```

```

        collapse = "; "
    )
}
)

group_feature_counts <- table(
  sers_sgl_selection$group
)

sers_selected_regions$n_selected_wavenumbers <- as.integer(
  group_feature_counts[
    as.character(sers_selected_regions$group)
  ]
)

sers_selected_regions <- sers_selected_regions[
  order(sers_selected_regions>window_start),
  ,
  drop = FALSE
]

cat(
  "Selected spectral regions:",
  nrow(sers_selected_regions),
  "\n"
)

print(sers_selected_regions)
}

```

```

##
## Running msgl (dense design matrix)
##
## Samples: Features: Classes: Groups: Parameters:
##      522      2.202k          2      112      4.404k
##
## Tied-model summary:
##   model_index  cv_error selected_wavenumbers
## 1           82 0.03448276              179
## 2           83 0.03448276              179
## 3           84 0.03448276              179
## Selected SGL model: 82
## CV error: 0.03448276
## Selected SERS wavenumbers: 179
##   [1] "SERS_650.0" "SERS_650.5" "SERS_651.0" "SERS_651.5" "SERS_652.0"
##   [6] "SERS_652.5" "SERS_653.0" "SERS_653.5" "SERS_654.0" "SERS_654.5"
##  [11] "SERS_655.0" "SERS_655.5" "SERS_656.0" "SERS_656.5" "SERS_657.0"
##  [16] "SERS_657.5" "SERS_658.0" "SERS_658.5" "SERS_659.0" "SERS_800.0"
##  [21] "SERS_800.5" "SERS_801.0" "SERS_801.5" "SERS_802.0" "SERS_802.5"
##  [26] "SERS_803.0" "SERS_803.5" "SERS_804.0" "SERS_804.5" "SERS_805.0"
##  [31] "SERS_805.5" "SERS_806.0" "SERS_806.5" "SERS_807.0" "SERS_807.5"
##  [36] "SERS_808.0" "SERS_808.5" "SERS_809.0" "SERS_809.5" "SERS_960.0"

```

```

## [41] "SERS_960.5" "SERS_961.0" "SERS_961.5" "SERS_962.0" "SERS_962.5"
## [46] "SERS_963.0" "SERS_963.5" "SERS_964.0" "SERS_964.5" "SERS_965.0"
## [51] "SERS_965.5" "SERS_966.0" "SERS_966.5" "SERS_967.0" "SERS_967.5"
## [56] "SERS_968.0" "SERS_968.5" "SERS_969.0" "SERS_969.5" "SERS_970.0"
## [61] "SERS_970.5" "SERS_971.0" "SERS_971.5" "SERS_972.0" "SERS_972.5"
## [66] "SERS_973.0" "SERS_973.5" "SERS_974.0" "SERS_974.5" "SERS_975.0"
## [71] "SERS_975.5" "SERS_976.0" "SERS_976.5" "SERS_977.0" "SERS_977.5"
## [76] "SERS_978.0" "SERS_978.5" "SERS_979.0" "SERS_979.5" "SERS_1020.0"
## [81] "SERS_1020.5" "SERS_1021.0" "SERS_1021.5" "SERS_1022.0" "SERS_1022.5"
## [86] "SERS_1023.0" "SERS_1023.5" "SERS_1024.0" "SERS_1024.5" "SERS_1025.0"
## [91] "SERS_1025.5" "SERS_1026.0" "SERS_1026.5" "SERS_1027.0" "SERS_1027.5"
## [96] "SERS_1028.0" "SERS_1028.5" "SERS_1029.0" "SERS_1029.5" "SERS_1510.0"
## [101] "SERS_1510.5" "SERS_1511.0" "SERS_1511.5" "SERS_1512.0" "SERS_1512.5"
## [106] "SERS_1513.0" "SERS_1513.5" "SERS_1514.0" "SERS_1514.5" "SERS_1515.0"
## [111] "SERS_1515.5" "SERS_1516.0" "SERS_1516.5" "SERS_1517.0" "SERS_1517.5"
## [116] "SERS_1518.0" "SERS_1518.5" "SERS_1519.0" "SERS_1519.5" "SERS_1540.0"
## [121] "SERS_1540.5" "SERS_1541.0" "SERS_1541.5" "SERS_1542.0" "SERS_1542.5"
## [126] "SERS_1543.0" "SERS_1543.5" "SERS_1544.0" "SERS_1544.5" "SERS_1545.0"
## [131] "SERS_1545.5" "SERS_1546.0" "SERS_1546.5" "SERS_1547.0" "SERS_1547.5"
## [136] "SERS_1548.0" "SERS_1548.5" "SERS_1549.0" "SERS_1549.5" "SERS_1610.0"
## [141] "SERS_1610.5" "SERS_1611.0" "SERS_1611.5" "SERS_1612.0" "SERS_1612.5"
## [146] "SERS_1613.0" "SERS_1613.5" "SERS_1614.0" "SERS_1614.5" "SERS_1615.0"
## [151] "SERS_1615.5" "SERS_1616.0" "SERS_1616.5" "SERS_1617.0" "SERS_1617.5"
## [156] "SERS_1618.0" "SERS_1618.5" "SERS_1619.0" "SERS_1619.5" "SERS_1640.0"
## [161] "SERS_1640.5" "SERS_1641.0" "SERS_1641.5" "SERS_1642.0" "SERS_1642.5"
## [166] "SERS_1643.0" "SERS_1643.5" "SERS_1644.0" "SERS_1644.5" "SERS_1645.0"
## [171] "SERS_1645.5" "SERS_1646.0" "SERS_1646.5" "SERS_1647.0" "SERS_1647.5"
## [176] "SERS_1648.0" "SERS_1648.5" "SERS_1649.0" "SERS_1649.5"
## Selected spectral regions: 9
##   group window_start window_end
## 1     11           650         660
## 2     26           800         810
## 3     42           960         970
## 4     43           970         980
## 5     48          1020        1030
## 6     97          1510        1520
## 7    100          1540        1550
## 8    107          1610        1620
## 9    110          1640        1650
##
## 1                                650; 650.5; 651; 651.5; 652; 652.5; 653; 653.5; 654; 654.5; 655; 655.5;
## 2                                800; 800.5; 801; 801.5; 802; 802.5; 803; 803.5; 804; 804.5; 805; 805.5; 806; 806.5;
## 3                                960; 960.5; 961; 961.5; 962; 962.5; 963; 963.5; 964; 964.5; 965; 965.5; 966; 966.5;
## 4                                970; 970.5; 971; 971.5; 972; 972.5; 973; 973.5; 974; 974.5; 975; 975.5; 976; 976.5;
## 5 1020; 1020.5; 1021; 1021.5; 1022; 1022.5; 1023; 1023.5; 1024; 1024.5; 1025; 1025.5; 1026; 1026.5;
## 6 1510; 1510.5; 1511; 1511.5; 1512; 1512.5; 1513; 1513.5; 1514; 1514.5; 1515; 1515.5; 1516; 1516.5;
## 7 1540; 1540.5; 1541; 1541.5; 1542; 1542.5; 1543; 1543.5; 1544; 1544.5; 1545; 1545.5; 1546; 1546.5;
## 8 1610; 1610.5; 1611; 1611.5; 1612; 1612.5; 1613; 1613.5; 1614; 1614.5; 1615; 1615.5; 1616; 1616.5;
## 9 1640; 1640.5; 1641; 1641.5; 1642; 1642.5; 1643; 1643.5; 1644; 1644.5; 1645; 1645.5; 1646; 1646.5;
##   n_selected_wavenumbers
## 1                        19
## 2                        20
## 3                        20
## 4                        20

```

```
## 5          20
## 6          20
## 7          20
## 8          20
## 9          20
```

Select spectral windows using a genetic algorithm

The GA searches among the same contiguous Raman windows used by SGL. Its fitness is the mean balanced accuracy from replicate-aware cross-validation minus a small penalty for the number of selected regions.

```
if (sers_selection_method == "GA") {

  set.seed(123)

  ga_min_regions <- 2L
  ga_max_regions <- min(10L, length(unique(sers_grouping)))
  ga_region_penalty <- 0.002

  # Represent every fixed Raman window by its mean intensity.
  all_group_ids <- sort(unique(sers_grouping))

  sers_window_matrix <- vapply(
    all_group_ids,
    function(group_id) {
      rowMeans(
        sers_x_raw[, sers_grouping == group_id, drop = FALSE],
        na.rm = TRUE
      )
    },
    numeric(nrow(sers_x_raw))
  )

  sers_window_matrix <- as.matrix(sers_window_matrix)
  colnames(sers_window_matrix) <- paste0("window_", all_group_ids)

  balanced_accuracy <- function(observed, predicted) {
    class_recall <- vapply(
      levels(sers_classes),
      function(class_name) {
        class_rows <- observed == class_name
        mean(predicted[class_rows] == class_name)
      },
      numeric(1)
    )

    mean(class_recall)
  }

  ga_fitness <- function(solution) {
    selected_windows <- which(solution == 1)
    n_selected <- length(selected_windows)
```

```

if (
  n_selected < ga_min_regions ||
  n_selected > ga_max_regions
) {
  return(-1e6)
}

fold_scores <- vapply(
  sgl_cv_indices,
  function(test_rows) {
    train_rows <- setdiff(seq_len(nrow(sers_window_matrix)), test_rows)

    x_train <- sers_window_matrix[
      train_rows,
      selected_windows,
      drop = FALSE
    ]

    x_test <- sers_window_matrix[
      test_rows,
      selected_windows,
      drop = FALSE
    ]

    train_center <- colMeans(x_train)
    train_scale <- apply(x_train, 2, stats::sd)
    train_scale[!is.finite(train_scale) | train_scale == 0] <- 1

    x_train <- sweep(x_train, 2, train_center, "-")
    x_train <- sweep(x_train, 2, train_scale, "/")
    x_test <- sweep(x_test, 2, train_center, "-")
    x_test <- sweep(x_test, 2, train_scale, "/")

    class_centroids <- vapply(
      levels(sers_classes),
      function(class_name) {
        colMeans(
          x_train[
            sers_classes[train_rows] == class_name,
            ,
            drop = FALSE
          ]
        )
      },
      numeric(ncol(x_train))
    )

    class_centroids <- as.matrix(class_centroids)

    distances <- vapply(
      seq_len(ncol(class_centroids)),
      function(class_index) {
        rowSums(

```

```

        sweep(
          x_test,
          2,
          class_centroids[, class_index],
          "_")
      )^2
    )
  },
  numeric(nrow(x_test))
)

predicted <- factor(
  levels(sers_classes)[
    max.col(-distances, ties.method = "first")
  ],
  levels = levels(sers_classes)
)

balanced_accuracy(
  observed = sers_classes[test_rows],
  predicted = predicted
)
},
numeric(1)
)

mean(fold_scores) - ga_region_penalty * n_selected
}

# Start with valid solutions containing 2-10 regions. This avoids
# an initial population dominated by invalid large subsets.
ga_initial_population <- matrix(
  0,
  nrow = 100,
  ncol = ncol(sers_window_matrix)
)

for (population_row in seq_len(nrow(ga_initial_population))) {
  initial_size <- sample(
    ga_min_regions:ga_max_regions,
    size = 1
  )

  ga_initial_population[
    population_row,
    sample(seq_len(ncol(sers_window_matrix)), initial_size)
  ] <- 1
}

ga_fit <- GA::ga(
  type = "binary",
  fitness = ga_fitness,
  nBits = ncol(sers_window_matrix),

```

```

popSize = 100,
maxiter = 300,
run = 50,
elitism = 5,
pcrossover = 0.8,
pmutation = 0.1,
suggestions = ga_initial_population,
seed = 123,
monitor = FALSE
)

ga_solution <- ga_fit@solution

if (is.matrix(ga_solution)) {
  ga_solution <- ga_solution[1, ]
}

selected_group_ids <- all_group_ids[which(ga_solution == 1)]

if (
  length(selected_group_ids) < ga_min_regions ||
  length(selected_group_ids) > ga_max_regions
) {
  stop("The GA solution does not satisfy the region-count constraints.")
}

selected_idx <- which(sers_grouping %in% selected_group_ids)
sers_selected_features <- colnames(sers_x_raw)[selected_idx]
sers_active_grouping <- sers_grouping

sers_sgl_selection <- data.frame(
  feature = sers_selected_features,
  wavenumber = sers_wavenumbers_raw[selected_idx],
  group = sers_grouping[selected_idx],
  stringsAsFactors = FALSE
)

sers_sgl_selection$window_start <- min(sers_wavenumbers_raw) +
  (sers_sgl_selection$group - 1) * sgl_window_width

sers_sgl_selection$window_end <-
  sers_sgl_selection$window_start + sgl_window_width

sers_selected_regions <- aggregate(
  wavenumber ~ group + window_start + window_end,
  data = sers_sgl_selection,
  FUN = function(z) paste(round(sort(z), 2), collapse = "; ")
)

sers_selected_regions$n_selected_wavenumbers <- as.integer(
  table(sers_sgl_selection$group)[
    as.character(sers_selected_regions$group)
  ]
)

```

```

)

sers_selected_regions <- sers_selected_regions[
  order(sers_selected_regions$window_start),
  ,
  drop = FALSE
]

ga_summary <- data.frame(
  fitness = ga_fit@fitnessValue,
  mean_cv_balanced_accuracy = ga_fit@fitnessValue +
    ga_region_penalty * length(selected_group_ids),
  region_penalty = ga_region_penalty,
  selected_regions = length(selected_group_ids),
  stringsAsFactors = FALSE
)

cat("GA-selected spectral regions:", nrow(sers_selected_regions), "\n")
print(sers_selected_regions)
print(ga_summary)
}

```

Construct variable-width windows using adjacent correlations

The WS method groups consecutive Raman shifts into the same window when their Pearson correlation across spectra is at least the selected threshold. Every variable-width window is represented by its mean intensity before aggregation to the six biological samples. This grouping is unsupervised and does not use the cell-line labels.

```

if (sers_selection_method == "WS") {

  adjacent_correlations <- vapply(
    seq_len(ncol(sers_x_raw) - 1L),
    function(index) {
      stats::cor(
        sers_x_raw[, index],
        sers_x_raw[, index + 1L],
        method = "pearson",
        use = "pairwise.complete.obs"
      )
    },
    numeric(1)
  )

  join_adjacent <- is.finite(adjacent_correlations) &
    adjacent_correlations >= ws_correlation_threshold

  # Start a new window whenever the adjacent correlation falls
  # below the selected threshold.
  ws_grouping <- cumsum(
    c(1L, as.integer(!join_adjacent))
  )
}

```

```

if (length(ws_grouping) != ncol(sers_x_raw)) {
  stop("WS grouping does not match the number of SERS variables.")
}

if (length(unique(ws_grouping)) < 2L) {
  stop(
    paste(
      "WS produced fewer than two windows.",
      "Increase ws_correlation_threshold."
    )
  )
}

sers_selected_features <- colnames(sers_x_raw)
sers_active_grouping <- ws_grouping
selected_idx <- seq_len(ncol(sers_x_raw))

ws_group_ids <- sort(unique(ws_grouping))

ws_window_start <- vapply(
  ws_group_ids,
  function(group_id) {
    min(sers_wavenumbers_raw[ws_grouping == group_id])
  },
  numeric(1)
)

ws_window_end <- vapply(
  ws_group_ids,
  function(group_id) {
    max(sers_wavenumbers_raw[ws_grouping == group_id])
  },
  numeric(1)
)

sers_sgl_selection <- data.frame(
  feature = sers_selected_features,
  wavenumber = sers_wavenumbers_raw,
  group = ws_grouping,
  window_start = ws_window_start[
    match(ws_grouping, ws_group_ids)
  ],
  window_end = ws_window_end[
    match(ws_grouping, ws_group_ids)
  ],
  stringsAsFactors = FALSE
)

sers_selected_regions <- data.frame(
  group = ws_group_ids,
  window_start = ws_window_start,
  window_end = ws_window_end,
  wavenumber = vapply(

```

```

ws_group_ids,
function(group_id) {
  paste(
    round(
      sers_wavenumbers_raw[ws_grouping == group_id],
      2
    ),
    collapse = "; "
  )
},
character(1)
),
n_selected_wavenumbers = as.integer(table(ws_grouping)),
stringsAsFactors = FALSE
)

ws_adjacent_correlations <- data.frame(
  lower_wavenumber = sers_wavenumbers_raw[
    seq_len(length(sers_wavenumbers_raw) - 1L)
  ],
  upper_wavenumber = sers_wavenumbers_raw[
    seq.int(2L, length(sers_wavenumbers_raw))
  ],
  correlation = adjacent_correlations,
  joined_into_same_window = join_adjacent,
  stringsAsFactors = FALSE
)

ws_summary <- data.frame(
  correlation_threshold = ws_correlation_threshold,
  number_of_windows = length(ws_group_ids),
  minimum_window_variables = min(table(ws_grouping)),
  median_window_variables = stats::median(table(ws_grouping)),
  maximum_window_variables = max(table(ws_grouping)),
  stringsAsFactors = FALSE
)

cat("WS variable-width spectral regions:", nrow(sers_selected_regions), "\n")
print(sers_selected_regions)
print(ws_summary)

# Display how the threshold determines the WS boundaries.
plot(
  ws_adjacent_correlations$lower_wavenumber,
  ws_adjacent_correlations$correlation,
  type = "l",
  xlab = expression(paste("Lower adjacent Raman shift (", cm^{-1}, ")")),
  ylab = "Pearson correlation",
  main = "Adjacent-wavenumber correlations used by WS"
)
abline(
  h = ws_correlation_threshold,
  col = "red",

```

```

    lty = 2,
    lwd = 2
  )

  # Display the number of measured wavenumbers assigned to each window.
  barplot(
    sers_selected_regions$n_selected_wavenumbers,
    names.arg = sers_selected_regions$group,
    xlab = "WS window",
    ylab = "Number of wavenumbers",
    main = "Widths of correlation-based SERS windows",
    border = NA,
    col = "steelblue"
  )
}

```

Export the SERS selection results

```

if (sers_selection_method == "SGL") {

write.table(
  sers_sgl_selection,
  file = result_file("selected_SERS_wavenumbers.tsv"),
  sep = "\t",
  quote = FALSE,
  row.names = FALSE
)

write.table(
  sers_selected_regions,
  file = result_file("selected_SERS_regions.tsv"),
  sep = "\t",
  quote = FALSE,
  row.names = FALSE
)

write.table(
  candidate_summary,
  file = result_file("tied_best_models.tsv"),
  sep = "\t",
  quote = FALSE,
  row.names = FALSE
)

sgl_cv_summary <- data.frame(
  model_index = seq_along(sgl_error),
  cv_error = sgl_error,
  stringsAsFactors = FALSE
)

write.table(
  sgl_cv_summary,

```

```

file = result_file("cross_validation_error_path.tsv"),
sep = "\t",
quote = FALSE,
row.names = FALSE
)

saveRDS(
  sgl_cv,
  result_file("cross_validation.rds")
)

saveRDS(
  sgl_fit,
  result_file("SERS_selection_model.rds")
)

}

if (sers_selection_method == "GA") {

  write.table(
    sers_sgl_selection,
    file = result_file("selected_SERS_wavenumbers.tsv"),
    sep = "\t",
    quote = FALSE,
    row.names = FALSE
  )

  write.table(
    sers_selected_regions,
    file = result_file("selected_SERS_regions.tsv"),
    sep = "\t",
    quote = FALSE,
    row.names = FALSE
  )

  write.table(
    ga_summary,
    file = result_file("selection_summary.tsv"),
    sep = "\t",
    quote = FALSE,
    row.names = FALSE
  )

  saveRDS(
    ga_fit,
    result_file("SERS_selection_model.rds")
  )
}

if (sers_selection_method == "WS") {

  write.table(

```

```

sers_sgl_selection,
file = result_file("selected_SERS_wavenumbers.tsv"),
sep = "\t",
quote = FALSE,
row.names = FALSE
)

write.table(
sers_selected_regions,
file = result_file("selected_SERS_regions.tsv"),
sep = "\t",
quote = FALSE,
row.names = FALSE
)

write.table(
ws_adjacent_correlations,
file = result_file("adjacent_wavenumber_correlations.tsv"),
sep = "\t",
quote = FALSE,
row.names = FALSE
)

write.table(
ws_summary,
file = result_file("selection_summary.tsv"),
sep = "\t",
quote = FALSE,
row.names = FALSE
)
}

```

Aggregate selected spectral regions by biological sample

```

selected_idx <- match(
sers_selected_features,
colnames(sers_x_raw)
)

selected_groups <- sers_active_grouping[selected_idx]

sers_x_selected <- sers_x_raw[
,
sers_selected_features,
drop = FALSE
]

# Average selected wavenumbers belonging to the same selected window
sers_region_matrix <- sapply(
sort(unique(selected_groups)),
function(group_id) {

```

```

group_columns <- which(selected_groups == group_id)

rowMeans(
  sers_x_selected[, group_columns, drop = FALSE],
  na.rm = TRUE
)
}
)

sers_region_matrix <- as.matrix(sers_region_matrix)

selected_group_ids <- sort(unique(selected_groups))

# Obtain the method-specific fixed or variable window boundaries.
region_match <- match(
  selected_group_ids,
  sers_selected_regions$group
)

if (anyNA(region_match)) {
  stop("Could not match selected groups to their spectral boundaries.")
}

region_start <- sers_selected_regions$window_start[region_match]
region_end <- sers_selected_regions$window_end[region_match]

colnames(sers_region_matrix) <- paste0(
  "SERS_region_",
  round(region_start, 1),
  "_",
  round(region_end, 1)
)

cat(
  paste("Individual", sers_selection_method, "selected wavenumbers:"),
  ncol(sers_x_selected),
  "\n"
)

```

Individual SGL selected wavenumbers: 179

```

cat(
  paste(sers_selection_method, "spectral regions after aggregation:"),
  ncol(sers_region_matrix),
  "\n"
)

```

SGL spectral regions after aggregation: 9

Aggregate spectra to six biological samples

```

sers_sum <- rowsum(

```

```

    sers_region_matrix,
    group = sers_sample_id,
    reorder = FALSE
)

sers_n <- table(sers_sample_id)

sers_mat <- sweep(
  sers_sum,
  MARGIN = 1,
  STATS = as.numeric(sers_n[rownames(sers_sum)]),
  FUN = "/"
)

sers_mat <- sers_mat[
  expected_samples,
  ,
  drop = FALSE
]

stopifnot(
  nrow(sers_mat) == 6,
  ncol(sers_mat) > 1,
  !anyNA(sers_mat),
  all(is.finite(sers_mat))
)

cat("Region-level SERS matrix supplied to DIABLO:\n")

```

```
## Region-level SERS matrix supplied to DIABLO:
```

```
print(dim(sers_mat))
```

```
## [1] 6 9
```

```
print(colnames(sers_mat))
```

```
## [1] "SERS_region_650_660" "SERS_region_800_810" "SERS_region_960_970"
## [4] "SERS_region_970_980" "SERS_region_1020_1030" "SERS_region_1510_1520"
## [7] "SERS_region_1540_1550" "SERS_region_1610_1620" "SERS_region_1640_1650"
```

Match samples and define group information across all omics datasets.

```
# Match samples across all omics datasets
```

```
common_samples <- Reduce(
  intersect,
```

```

list(
  colnames(rna_counts),
  colnames(prot_mat),
  rownames(sers_mat)
)
)

common_samples <- expected_samples[
  expected_samples %in% common_samples
]

stopifnot(identical(common_samples, expected_samples))

rna_counts <- rna_counts[, common_samples, drop = FALSE]
prot_mat <- prot_mat[, common_samples, drop = FALSE]
sers_mat <- sers_mat[common_samples, , drop = FALSE]

sample_info <- data.frame(
  sample_id = common_samples,
  group = factor(
    get_group(common_samples),
    levels = c("1G", "DLN")
  ),
  biological_replicate = factor(
    get_biorep(common_samples),
    levels = c("A", "B", "C")
  )
)
)

write.table(
  sample_info,
  result_file("sample_info_from_names.tsv"),
  sep = "\t",
  quote = FALSE,
  row.names = FALSE
)

Y <- sample_info$group

stopifnot(
  length(levels(droplevels(Y))) == 2
)

sample_info

```

```

##   sample_id group biological_replicate
## 1      1G_A   1G                A
## 2      1G_B   1G                B
## 3      1G_C   1G                C
## 4     DLN_A  DLN                A
## 5     DLN_B  DLN                B
## 6     DLN_C  DLN                C

```

```
summary(Y)
```

```
## 1G DLN  
## 3 3
```

Filter and transform the RNA-seq data

```
# Filter and variance-stabilize the RNA-seq data
```

```
keep_gene <- filterByExpr(  
  rna_counts,  
  group = Y  
)  
  
rna_counts_f <- rna_counts[  
  keep_gene,  
  ,  
  drop = FALSE  
)  
  
dds <- DESeqDataSetFromMatrix(  
  countData = rna_counts_f,  
  colData = data.frame(  
    group = Y,  
    row.names = colnames(rna_counts_f)  
  ),  
  design = ~ group  
)  
  
dds <- estimateSizeFactors(dds)  
  
rna_vst <- assay(  
  vst(dds, blind = FALSE)  
)  
  
rna_block <- t(rna_vst)  
rna_block <- as.matrix(rna_block)  
  
stopifnot(  
  ncol(rna_block) > 1,  
  !anyNA(rna_block),  
  all(is.finite(rna_block))  
)  
  
dim(rna_block)
```

```
## [1] 6 23880
```

```
rna_block[  
  1:min(5, nrow(rna_block)),
```

```
1:min(5, ncol(rna_block))
]
```

```
##      ENSG00000223972 ENSG00000227232 ENSG00000237683 ENSG00000241860
## 1G_A      6.480911      9.988404      6.874668      7.745292
## 1G_B      6.292745      9.775306      6.536056      8.021999
## 1G_C      6.490082      9.996286      6.807511      7.864936
## DLN_A      6.447536      9.838561      6.506643      7.682277
## DLN_B      6.284856      10.071919      6.642632      7.818799
##      ENSG00000228463
## 1G_A      7.049664
## 1G_B      6.937646
## 1G_C      7.068778
## DLN_A      6.868038
## DLN_B      7.126256
```

Transform the proteomics data

```
# Log2-transform the raw LFQ intensities

if (any(prot_mat <= 0, na.rm = TRUE)) {
  stop(
    "Proteomics matrix contains non-positive values after missing-value filtering."
  )
}

prot_block <- log2(t(prot_mat))
prot_block <- as.matrix(prot_block)

colnames(prot_block) <- make.unique(
  colnames(prot_block)
)

stopifnot(
  ncol(prot_block) > 1,
  !anyNA(prot_block),
  all(is.finite(prot_block))
)

dim(prot_block)
```

```
## [1]      6 2470
```

```
prot_block[
  1:min(5, nrow(prot_block)),
  1:min(5, ncol(prot_block))
]
```

```
##      HDLBP      EHD1      YTHDF3      PSMA2      DIABLO
## 1G_A 28.84604 27.04562 25.25688 27.55223 22.16125
```

```
## 1G_B 29.05434 26.02886 25.45133 28.09627 21.60449
## 1G_C 28.57726 26.52268 25.03762 27.40088 20.10162
## DLN_A 29.14273 25.48867 24.03389 28.63613 23.95447
## DLN_B 28.97282 26.09782 24.80622 27.99847 23.05245
```

Prepare the SERS data

```
# Prepare the preprocessed SERS region matrix
```

```
sers_block <- as.matrix(sers_mat)
storage.mode(sers_block) <- "double"
```

```
colnames(sers_block) <- make.unique(
  colnames(sers_block)
)
```

```
stopifnot(
  ncol(sers_block) > 1,
  !anyNA(sers_block),
  all(is.finite(sers_block))
)
```

```
dim(sers_block)
```

```
## [1] 6 9
```

```
sers_block[
  1:min(5, nrow(sers_block)),
  1:min(5, ncol(sers_block))
]
```

```
##      SERS_region_650_660 SERS_region_800_810 SERS_region_960_970
## 1G_A      0.03235299      0.007800860      0.012302768
## 1G_B      0.03103732      0.007266787      0.012004705
## 1G_C      0.03420176      0.007969825      0.011339814
## DLN_A      0.04904223      0.016798207      0.006579865
## DLN_B      0.04949328      0.016518232      0.006050852
##      SERS_region_970_980 SERS_region_1020_1030
## 1G_A      0.010101051      0.006059632
## 1G_B      0.009294148      0.005995136
## 1G_C      0.009824774      0.005406303
## DLN_A      0.005127409      0.013073970
## DLN_B      0.004415159      0.012411635
```

Map RNA Ensembl identifiers to gene symbols

```
# Map Ensembl identifiers to gene symbols
```

```
rna_feature_original <- colnames(rna_block)
```

```

rna_feature_clean <- trimws(
  sub("\\.[0-9]+$", "", rna_feature_original)
)

is_ensembl <- grepl(
  "^ENSG[0-9]+$",
  rna_feature_clean
)

gene_symbols <- rep(
  NA_character_,
  length(rna_feature_clean)
)

if (any(is_ensembl)) {
  gene_symbols[is_ensembl] <- as.character(
    AnnotationDbi::mapIds(
      org.Hs.eg.db,
      keys = rna_feature_clean[is_ensembl],
      keytype = "ENSEMBL",
      column = "SYMBOL",
      multiVals = "first"
    )
  )
}

rna_labels <- ifelse(
  !is.na(gene_symbols) & gene_symbols != "",
  gene_symbols,
  rna_feature_clean
)

colnames(rna_block) <- make.unique(rna_labels)

rna_annotation <- data.frame(
  original_feature = rna_feature_original,
  cleaned_feature = rna_feature_clean,
  is_ensembl = is_ensembl,
  gene_symbol = gene_symbols,
  diablo_label = colnames(rna_block),
  stringsAsFactors = FALSE
)

write.table(
  rna_annotation,
  result_file("DIABLO_RNA_gene_annotation.tsv"),
  sep = "\t",
  quote = FALSE,
  row.names = FALSE
)

cat(
  "Total RNA features:",

```

```
nrow(rna_annotation),
"\n"
)
```

```
## Total RNA features: 23880
```

```
cat(
  "Features mapped to gene symbols:",
  sum(!is.na(gene_symbols)),
  "\n"
)
```

```
## Features mapped to gene symbols: 18045
```

```
cat(
  "Features retained as Ensembl IDs:",
  sum(is.na(gene_symbols)),
  "\n"
)
```

```
## Features retained as Ensembl IDs: 5835
```

```
head(rna_annotation)
```

```
##   original_feature cleaned_feature is_ensembl gene_symbol   diablo_label
## 1  ENSG00000223972 ENSG00000223972      TRUE      <NA> ENSG00000223972
## 2  ENSG00000227232 ENSG00000227232      TRUE      <NA> ENSG00000227232
## 3  ENSG00000237683 ENSG00000237683      TRUE      <NA> ENSG00000237683
## 4  ENSG00000241860 ENSG00000241860      TRUE      <NA> ENSG00000241860
## 5  ENSG00000228463 ENSG00000228463      TRUE  RPL23AP21    RPL23AP21
## 6  ENSG00000237094 ENSG00000237094      TRUE      <NA> ENSG00000237094
```

Feature pre-filtering and scaling

```
# The RNA block is reduced to 2,000 high-variance genes to limit the
# computational burden of parameter tuning. All complete proteins and all
# SERS regions are retained; DIABLO performs the final sparse feature selection.
```

```
top_rna <- 2000
top_prot <- Inf
top_sers <- Inf

filter_and_select <- function(
  x,
  n_features,
  block_name
) {
  x <- as.matrix(x)
  storage.mode(x) <- "double"
```

```

# Remove features containing non-finite values
finite_features <- apply(
  x,
  2,
  function(z) all(is.finite(z))
)

cat(
  block_name,
  "non-finite features removed:",
  sum(!finite_features),
  "\n"
)

x <- x[, finite_features, drop = FALSE]

# Calculate variance before scaling
feature_variance <- apply(
  x,
  2,
  var
)

variable_features <- is.finite(feature_variance) &
  feature_variance > 0

cat(
  block_name,
  "zero-variance features removed:",
  sum(!variable_features),
  "\n"
)

x <- x[, variable_features, drop = FALSE]
feature_variance <- feature_variance[variable_features]

stopifnot(ncol(x) > 1)

# Retain all features when n_features is infinite. Otherwise, retain the
# requested number of features with the highest unsupervised variance.
n_select <- min(
  n_features,
  ncol(x)
)

selected_features <- names(
  sort(
    feature_variance,
    decreasing = TRUE
  )
)[seq_len(n_select)]

x <- x[

```

```

      ,
      selected_features,
      drop = FALSE
    ]

    cat(
      block_name,
      "features retained:",
      ncol(x),
      "\n"
    )

    x
  }

rna_block <- filter_and_select(
  rna_block,
  top_rna,
  "RNA"
)

```

```

## RNA non-finite features removed: 0
## RNA zero-variance features removed: 0
## RNA features retained: 2000

```

```

prot_block <- filter_and_select(
  prot_block,
  top_prot,
  "Proteome"
)

```

```

## Proteome non-finite features removed: 0
## Proteome zero-variance features removed: 0
## Proteome features retained: 2470

```

```

sers_block <- filter_and_select(
  sers_block,
  top_sers,
  "SERS"
)

```

```

## SERS non-finite features removed: 0
## SERS zero-variance features removed: 0
## SERS features retained: 9

```

```

# Scale the selected features
rna_block <- as.matrix(
  scale(rna_block)
)

prot_block <- as.matrix(
  scale(prot_block)
)

```

```

)

sers_block <- as.matrix(
  scale(sers_block)
)

# Ensure unique feature labels
colnames(rna_block) <- make.unique(
  colnames(rna_block)
)

colnames(prot_block) <- make.unique(
  colnames(prot_block)
)

colnames(sers_block) <- make.unique(
  colnames(sers_block)
)

# Verify sample correspondence
stopifnot(
  identical(
    rownames(rna_block),
    expected_samples
  ),
  identical(
    rownames(prot_block),
    expected_samples
  ),
  identical(
    rownames(sers_block),
    expected_samples
  )
)

# Verify that all matrices are suitable for DIABLO
stopifnot(
  all(is.finite(rna_block)),
  all(is.finite(prot_block)),
  all(is.finite(sers_block)),
  nrow(rna_block) == length(Y),
  nrow(prot_block) == length(Y),
  nrow(sers_block) == length(Y),
  ncol(rna_block) > 1,
  ncol(prot_block) > 1,
  ncol(sers_block) > 1
)

data <- list(
  transcriptome = rna_block,
  proteome = prot_block,
  sers = sers_block
)

```

```
lapply(data, dim)
```

```
## $transcriptome
## [1] 6 2000
##
## $proteome
## [1] 6 2470
##
## $sers
## [1] 6 9
```

```
summary(Y)
```

```
## 1G DLN
## 3 3
```

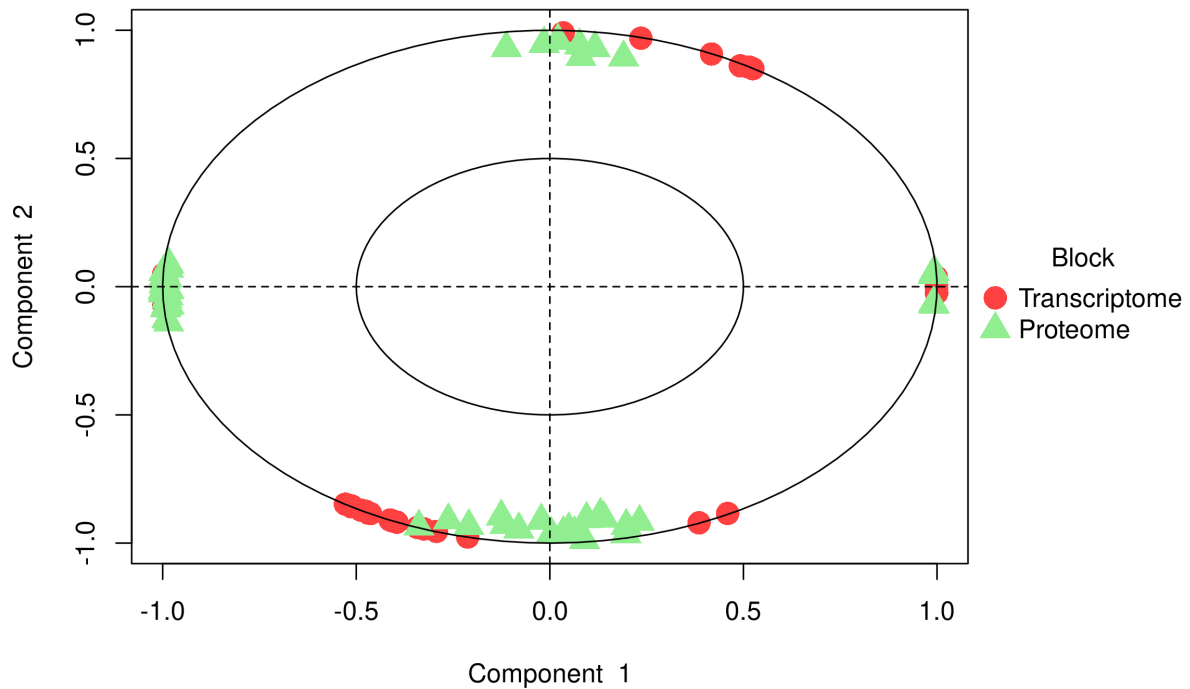
Pairwise sPLS comparisons between all omics blocks

```
sers_pairwise_keep <- rep(
  min(9L, ncol(data[["sers"]])),
  2
)

pls_rna_prot <- spls(
  X = data[["transcriptome"]],
  Y = data[["proteome"]],
  ncomp = 2,
  keepX = c(25, 25),
  keepY = c(25, 25)
)

plotVar(
  pls_rna_prot,
  cutoff = 0.5,
  title = "Transcriptome vs proteome",
  legend = c("Transcriptome", "Proteome"),
  var.names = FALSE,
  style = "graphics",
  pch = c(16, 17),
  cex = c(2, 2),
  col = c("brown1", "lightgreen")
)
```

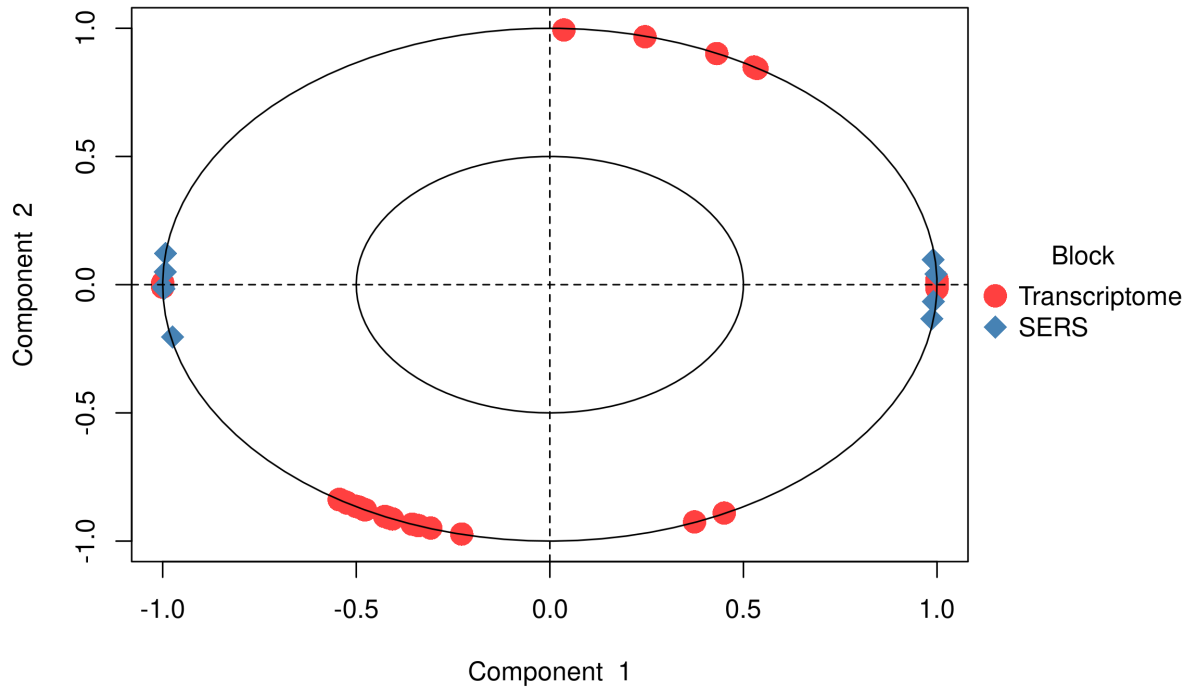
Transcriptome vs proteome



```
pls_rna_sers <- spls(
  X = data[["transcriptome"]],
  Y = data[["sers"]],
  ncomp = 2,
  keepX = c(25, 25),
  keepY = sers_pairwise_keep
)

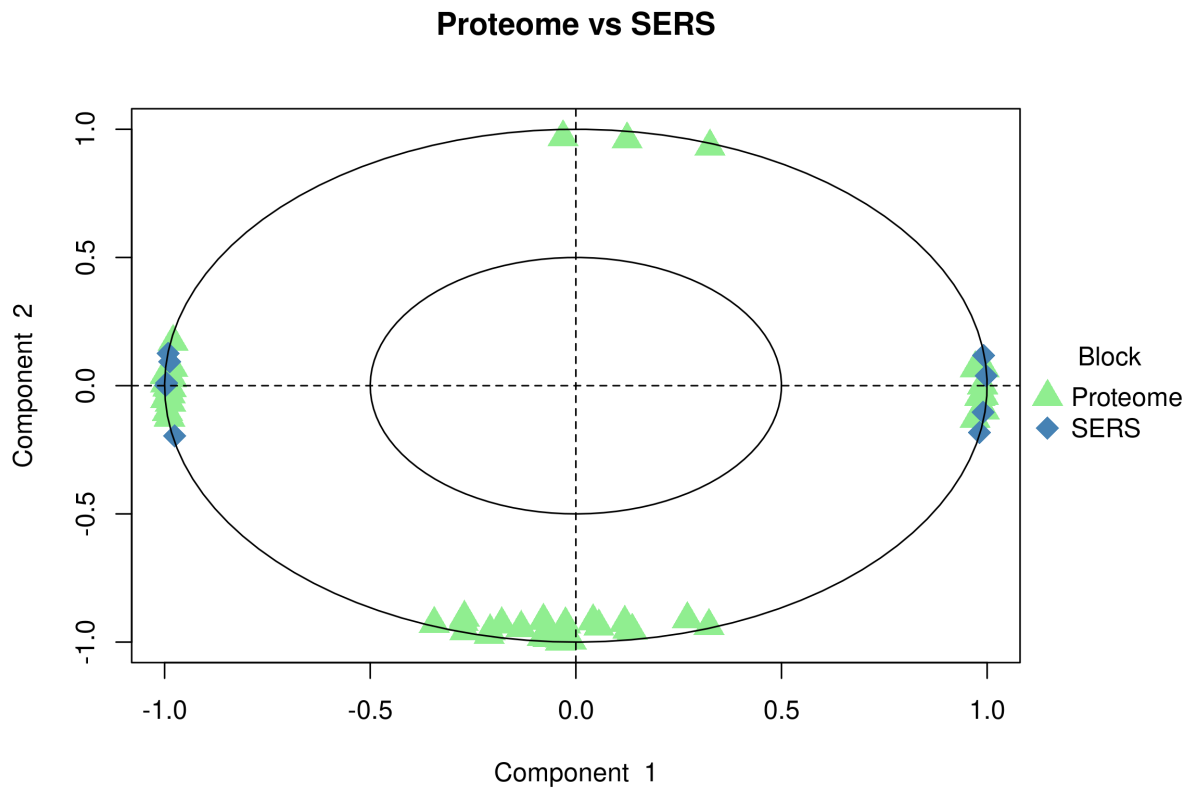
plotVar(
  pls_rna_sers,
  cutoff = 0.5,
  title = "Transcriptome vs SERS",
  legend = c("Transcriptome", "SERS"),
  var.names = FALSE,
  style = "graphics",
  pch = c(16, 18),
  cex = c(2, 2),
  col = c("brown1", "steelblue")
)
```

Transcriptome vs SERS



```
pls_prot_sers <- spls(
  X = data[["proteome"]],
  Y = data[["sers"]],
  ncomp = 2,
  keepX = c(25, 25),
  keepY = sers_pairwise_keep
)

plotVar(
  pls_prot_sers,
  cutoff = 0.5,
  title = "Proteome vs SERS",
  legend = c("Proteome", "SERS"),
  var.names = FALSE,
  style = "graphics",
  pch = c(17, 18),
  cex = c(2, 2),
  col = c("lightgreen", "steelblue")
)
```



Correlations between pairwise sPLS components

```
cor_rna_prot <- cor(
  pls_rna_prot$variates$X,
  pls_rna_prot$variates$Y
)

cor_rna_sers <- cor(
  pls_rna_sers$variates$X,
  pls_rna_sers$variates$Y
)

cor_prot_sers <- cor(
  pls_prot_sers$variates$X,
  pls_prot_sers$variates$Y
)

pairwise_component_correlations <- data.frame(
  comparison = c(
    "Transcriptome-proteome",
    "Transcriptome-SERS",
    "Proteome-SERS"
  )
)
```

```

),
component_1 = c(
  cor_rna_prot[1, 1],
  cor_rna_sers[1, 1],
  cor_prot_sers[1, 1]
),
component_2 = c(
  cor_rna_prot[2, 2],
  cor_rna_sers[2, 2],
  cor_prot_sers[2, 2]
)
)

pairwise_component_correlations

```

```

##           comparison component_1 component_2
## 1 Transcriptome-proteome  0.9995898  0.9991911
## 2   Transcriptome-SERS   0.9999883  0.8801560
## 3     Proteome-SERS     0.9987512  0.9981008

```

DIABLO model

Define the weakly connected DIABLO design matrix

```

design <- matrix(
  0.1,
  nrow = length(data),
  ncol = length(data),
  dimnames = list(
    names(data),
    names(data)
  )
)

diag(design) <- 0

design

```

```

##           transcriptome proteome sers
## transcriptome      0.0      0.1  0.1
## proteome          0.1      0.0  0.1
## sers              0.1      0.1  0.0

```

Fit the initial DIABLO model

```

ncomp_start <- 2

set.seed(123)

```

```

basic_diablo_model <- block.splsda(
  X = data,
  Y = Y,
  ncomp = ncomp_start,
  design = design,
  near.zero.var = TRUE
)

basic_diablo_model

```

```

##
## Call:
## block.splsda(X = data, Y = Y, ncomp = ncomp_start, design = design, near.zero.var = TRUE)
##
## sGCCA with 2 components on block 1 named transcriptome
## sGCCA with 2 components on block 2 named proteome
## sGCCA with 2 components on block 3 named sers
## sGCCA with 2 components on the outcome Y
##
## Dimension of block 1 is 6 2000
## Dimension of block 2 is 6 2470
## Dimension of block 3 is 6 9
## Outcome Y has 2 levels
##
## Selection of 2000 2000 variables on each of the sGCCA components on the block 1
## Selection of 2470 2470 variables on each of the sGCCA components on the block 2
## Selection of 9 9 variables on each of the sGCCA components on the block 3
##
## Main numerical outputs:
## -----
## loading vectors: see object$loadings
## variates: see object$variates
## variable names: see object$names
##
## Functions to visualise samples:
## -----
## plotIndiv, plotArrow, cimDiablo, plotDiablo
##
## Functions to visualise variables:
## -----
## plotVar, plotLoadings, network, circosPlot
##
## Other functions:
## -----
## selectVar, perf, auc

```

Evaluate the number of components

```

n_folds <- min(
  3,
  min(table(Y))
)

```

```

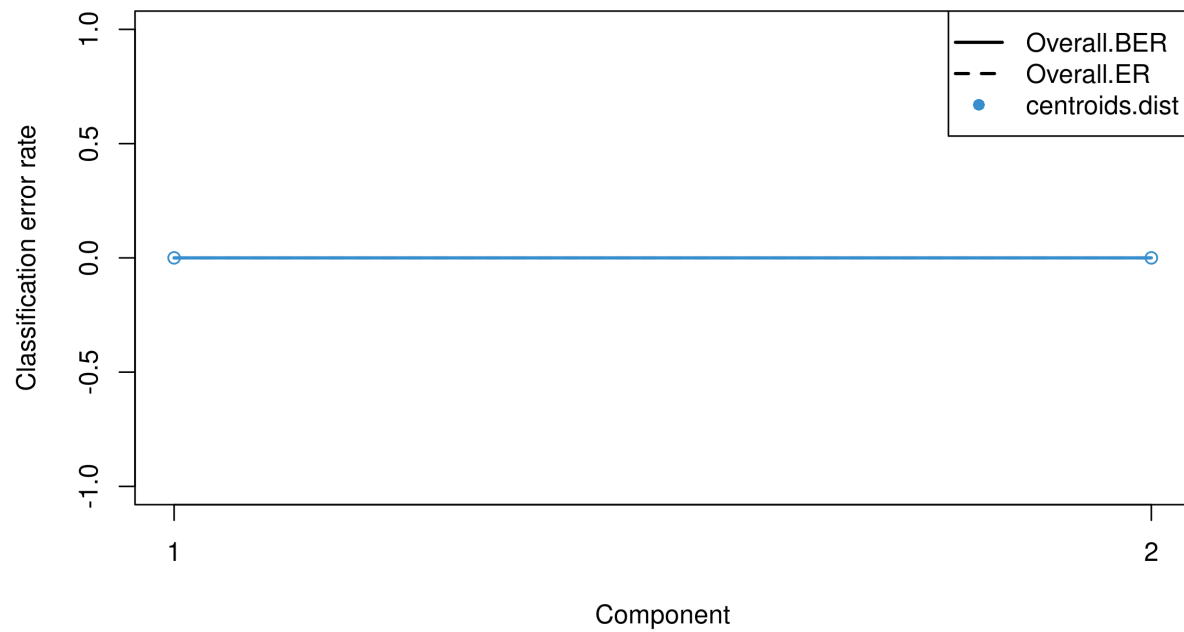
)

set.seed(123)

perf_initial <- perf(
  basic_diablo_model,
  validation = "Mfold",
  folds = n_folds,
  nrepeat = 10,
  dist = "centroids.dist",
  BPPARAM = BiocParallel::SerialParam()
)

plot(
  perf_initial,
  sd = TRUE
)

```



```
perf_initial$error.rate
```

```

## $transcriptome
##       centroids.dist
## comp1              0
## comp2              0
##
## $proteome
##       centroids.dist

```

```
## comp1          0
## comp2          0
##
## $sers
##      centroids.dist
## comp1          0
## comp2          0
```

Select the optimal number of components

```
ncomp_tuned <- perf_initial$choice.ncomp$WeightedVote[
  "Overall.BER",
  "centroids.dist"
]

ncomp_final <- as.numeric(ncomp_tuned)

stopifnot(
  is.finite(ncomp_final),
  ncomp_final >= 1,
  ncomp_final <= ncomp_start
)

cat(
  "Selected number of components:",
  ncomp_final,
  "\n"
)
```

```
## Selected number of components: 1
```

```
perf_initial$choice.ncomp$WeightedVote
```

```
##      centroids.dist
## Overall.ER         1
## Overall.BER        1
```

Tune the number of selected features

```
# Candidate signatures containing 5, 10, or 15 transcriptomic and proteomic
# features were evaluated. All nine SGL-selected SERS regions were retained
# for DIABLO integration.
test_keep_x <- list(
  transcriptome = c(5,10,15),
  proteome = c(5,10,15),
  sers = c(2, 5, 9)
)

# Ensure that candidates do not exceed the available features
```

```
test_keep_x <- Map(
  function(candidates, block) {
    candidates[candidates <= ncol(block)]
  },
  candidates = test_keep_x,
  block = data
)
```

```
set.seed(123)
```

```
tune_diablo <- tune.block.splsda(
  X = data,
  Y = Y,
  ncomp = ncomp_final,
  test.keepX = test_keep_x,
  design = design,
  validation = "Mfold",
  folds = n_folds,
  nrepeat = 10,
  dist = "centroids.dist",
  near.zero.var = TRUE,
  BPPARAM = BiocParallel::SerialParam()
)
```

```
tune_diablo$choice.keepX
```

```
## $transcriptome
## [1] 5
##
## $proteome
## [1] 5
##
## $sers
## [1] 2
```

```
keepx_tuning_summary <- do.call(
  rbind,
  lapply(
    names(tune_diablo$choice.keepX),
    function(block_name) {
      data.frame(
        block = block_name,
        component = seq_along(tune_diablo$choice.keepX[[block_name]]),
        selected_features = as.integer(
          tune_diablo$choice.keepX[[block_name]]
        ),
        stringsAsFactors = FALSE
      )
    }
  )
)

print(keepx_tuning_summary)
```

```
##           block component selected_features
## 1 transcriptome           1                5
## 2      proteome           1                5
## 3         sers            1                2
```

```
write.table(
  keepx_tuning_summary,
  file = result_file("DIABLO_tuned_keepX.tsv"),
  sep = "\t",
  quote = FALSE,
  row.names = FALSE
)
```

Extract the tuned feature numbers

```
keep_x_final <- tune_diablo$choice.keepX

keep_x_final
```

```
## $transcriptome
## [1] 5
##
## $proteome
## [1] 5
##
## $sers
## [1] 2
```

Final model

Fit the final sparse DIABLO model

```
set.seed(123)

final_diablo_model <- block.splsda(
  X = data,
  Y = Y,
  ncomp = ncomp_final,
  keepX = keep_x_final,
  design = design,
  near.zero.var = TRUE
)

saveRDS(
  final_diablo_model,
  result_file("DIABLO_final_model.rds")
)

final_diablo_model
```

```
##
## Call:
## block.splsda(X = data, Y = Y, ncomp = ncomp_final, keepX = keep_x_final, design = design, near.zero
##
## sGCCA with 1 components on block 1 named transcriptome
## sGCCA with 1 components on block 2 named proteome
## sGCCA with 1 components on block 3 named sers
## sGCCA with 1 components on the outcome Y
##
## Dimension of block 1 is 6 2000
## Dimension of block 2 is 6 2470
## Dimension of block 3 is 6 9
## Outcome Y has 2 levels
##
## Selection of 5 variables on each of the sGCCA components on the block 1
## Selection of 5 variables on each of the sGCCA components on the block 2
## Selection of 2 variables on each of the sGCCA components on the block 3
##
## Main numerical outputs:
## -----
## loading vectors: see object$loadings
## variates: see object$variates
## variable names: see object$names
##
## Functions to visualise samples:
## -----
## plotIndiv, plotArrow, cimDiablo, plotDiablo
##
## Functions to visualise variables:
## -----
## plotVar, plotLoadings, network, circosPlot
##
## Other functions:
## -----
## selectVar, perf, auc
```

Display the design matrix used in the final model

```
final_diablo_model$design
```

```
##           transcriptome proteome sers Y
## transcriptome          0.0      0.1 0.1 1
## proteome              0.1      0.0 0.1 1
## sers                  0.1      0.1 0.0 1
## Y                    1.0      1.0 1.0 0
```

Extract and export the selected DIABLO features

```
selected_features <- lapply(
  names(data),
```

```

function(block_name) {
  selected <- selectVar(
    final_diablo_model,
    block = block_name,
    comp = 1
  )[[block_name]]

  data.frame(
    block = block_name,
    component = 1,
    feature = selected$name,
    loading = as.numeric(selected$value[, 1]),
    stringsAsFactors = FALSE
  )
}
)

selected_features <- bind_rows(
  selected_features
)

write.table(
  selected_features,
  result_file("DIABLO_selected_features.tsv"),
  sep = "\t",
  quote = FALSE,
  row.names = FALSE
)

selected_features

```

##	block	component	feature	loading
## 1	transcriptome	1	ADORA1	-0.64516921
## 2	transcriptome	1	FAM133A	0.47755547
## 3	transcriptome	1	SPON2	-0.42607253
## 4	transcriptome	1	PSAPL1	-0.40494950
## 5	transcriptome	1	ENSG00000259886	-0.10087403
## 6	proteome	1	DNMT1	-0.65247044
## 7	proteome	1	ALDH1A3	0.52951585
## 8	proteome	1	COL1A2	0.40144543
## 9	proteome	1	ITGA2	0.36056838
## 10	proteome	1	COL18A1	0.05222355
## 11	sers	1	SERS_region_800_810	0.84475889
## 12	sers	1	SERS_region_1610_1620	0.53514710

Displaying DIABLO loading weights

```

short_label <- function(x, max_chars = 32) {
  x <- as.character(x)

  ifelse(

```

```

    nchar(x) > max_chars,
    paste0(
      substr(x, 1, max_chars - 3),
      "... "
    ),
    x
  )
}

plot_block_loadings <- function(
  model,
  block,
  comp = 1,
  top_n = 15,
  main = NULL
) {
  loadings <- model$loadings[[block]][, comp]

  loadings <- loadings[
    is.finite(loadings) & loadings != 0
  ]

  if (length(loadings) == 0) {
    stop(
      "No non-zero loadings were found for ",
      block,
      " on component ",
      comp,
      "."
    )
  }

  loadings <- loadings[
    order(
      abs(loadings),
      decreasing = TRUE
    )
  ]

  loadings <- head(
    loadings,
    top_n
  )

  loadings <- rev(loadings)

  names(loadings) <- short_label(
    names(loadings)
  )

  loading_colours <- ifelse(
    loadings > 0,
    "darkorange2",

```

```

    "dodgerblue3"
  )

old_par <- par(no.readonly = TRUE)
on.exit(par(old_par), add = TRUE)

par(
  mar = c(5, 12, 4, 2),
  cex = 0.85
)

barplot(
  loadings,
  horiz = TRUE,
  las = 1,
  cex.names = 1,
  col = loading_colours,
  border = "grey25",
  main = if (is.null(main)) {
    paste(
      block,
      "component",
      comp,
      "loadings"
    )
  } else {
    main
  },
  xlab = "Loading weight"
)

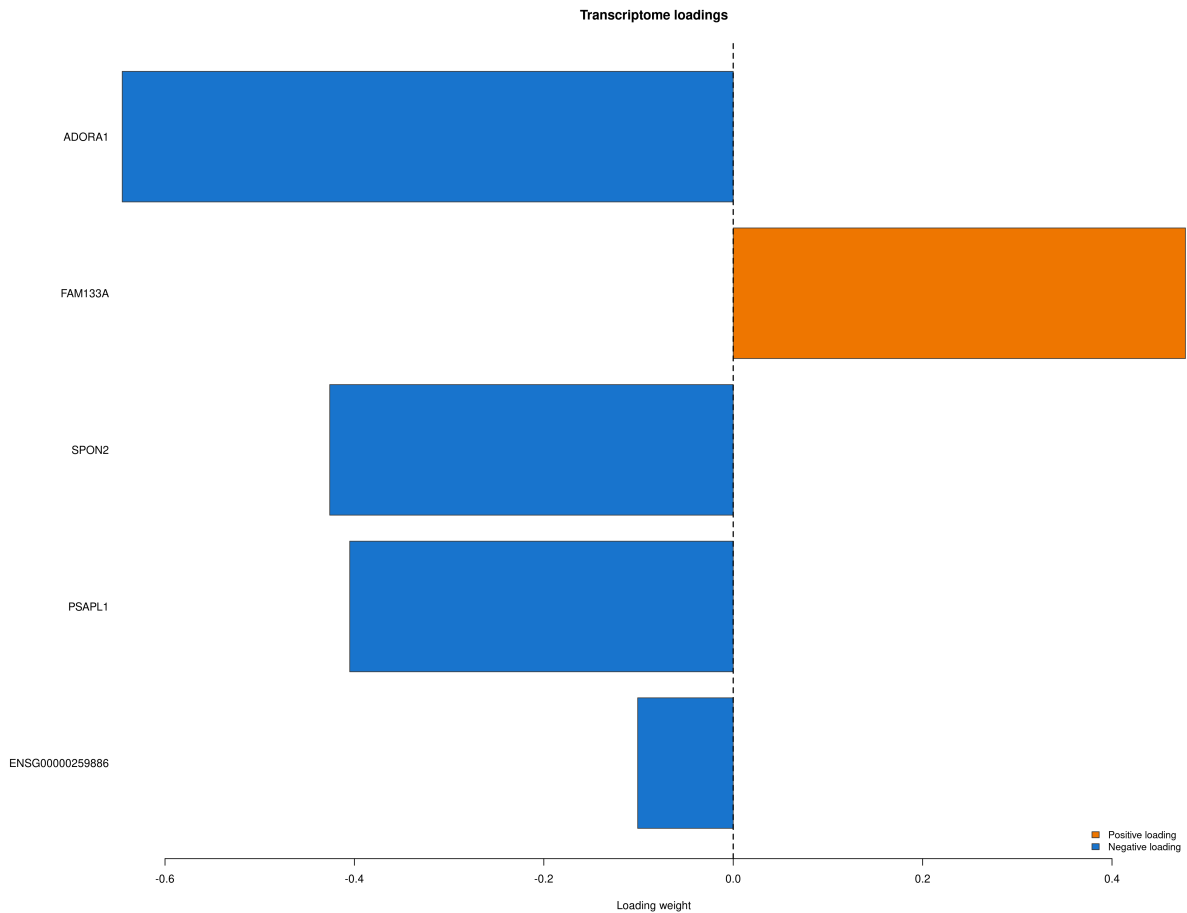
abline(
  v = 0,
  lty = 2,
  lwd = 1.5
)

legend(
  "bottomright",
  legend = c(
    "Positive loading",
    "Negative loading"
  ),
  fill = c(
    "darkorange2",
    "dodgerblue3"
  ),
  border = "grey25",
  bty = "n",
  cex = 0.9
)
}

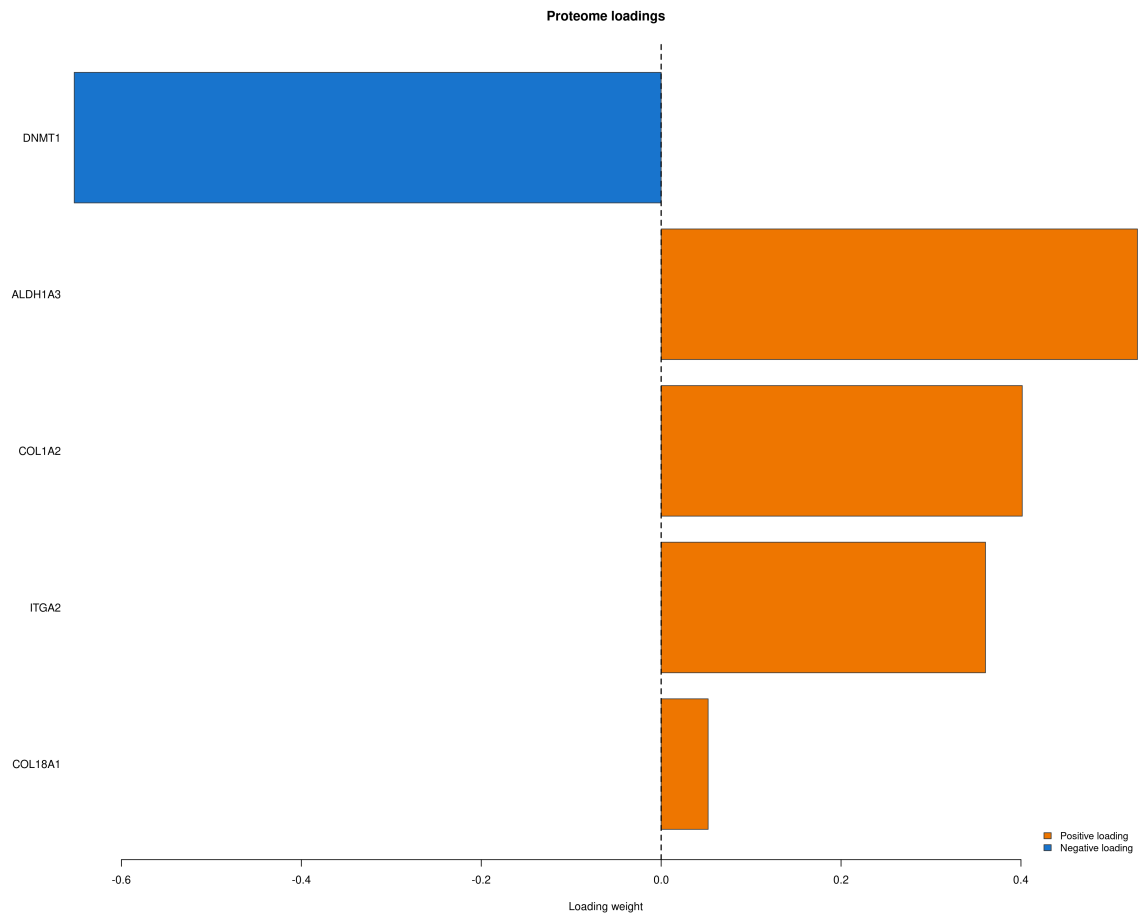
```

Plot loadings from the single selected component

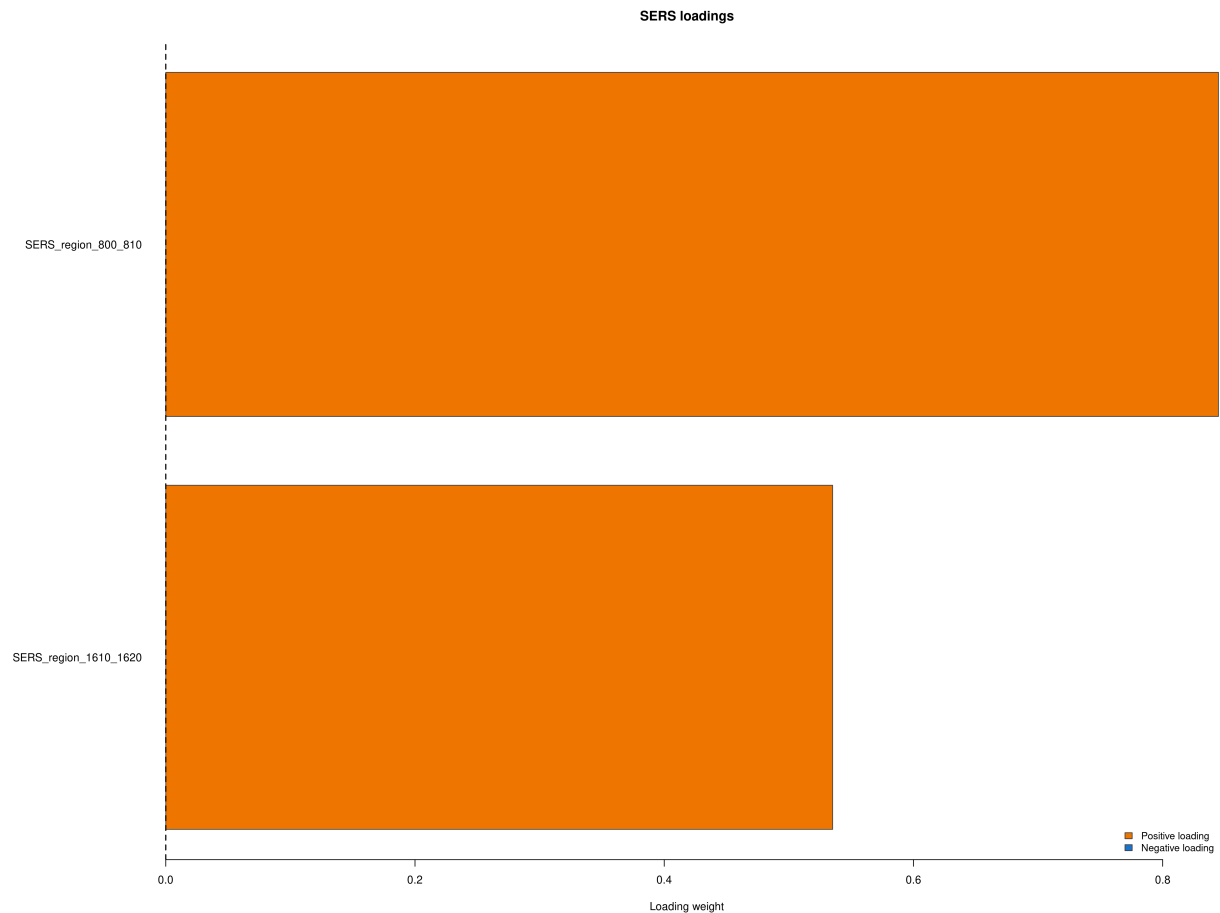
```
plot_block_loadings(  
  final_diablo_model,  
  block = "transcriptome",  
  comp = 1,  
  main = "Transcriptome loadings"  
)
```



```
plot_block_loadings(  
  final_diablo_model,  
  block = "proteome",  
  comp = 1,  
  main = "Proteome loadings"  
)
```



```
plot_block_loadings(  
  final_diablo_model,  
  block = "sers",  
  comp = 1,  
  main = "SERS loadings"  
)
```



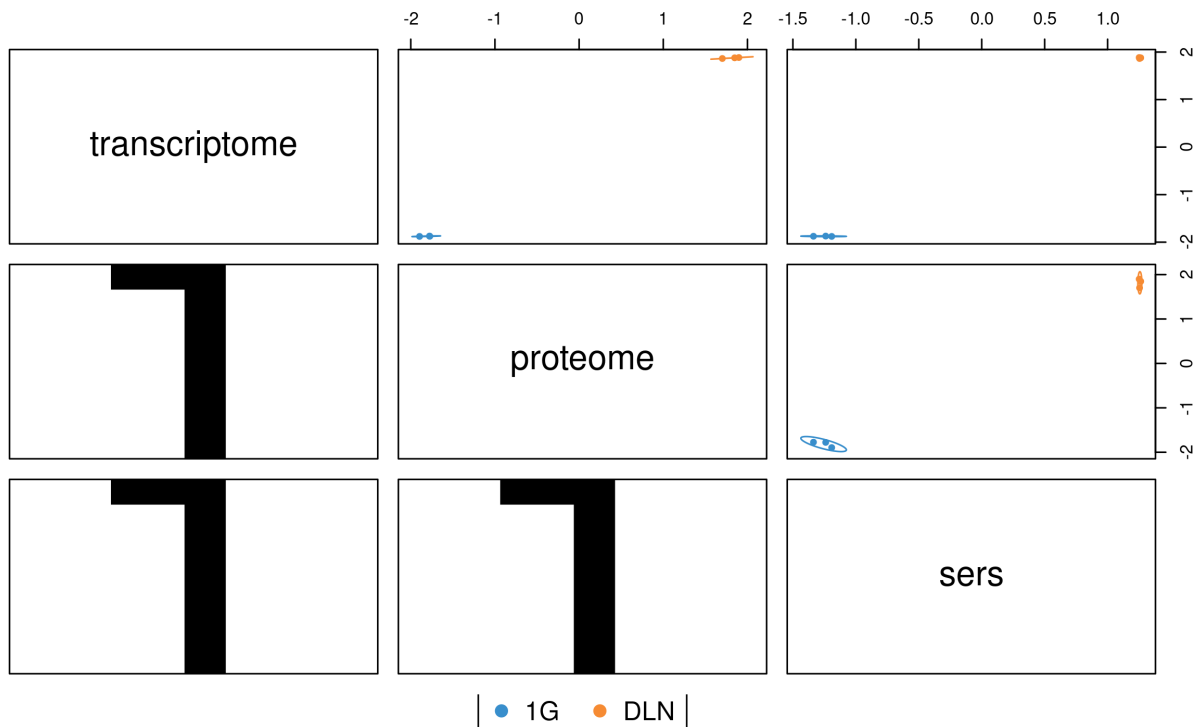
DIABLO visualizations

```
# Fit a two-component model for exploratory visualization

set.seed(123)

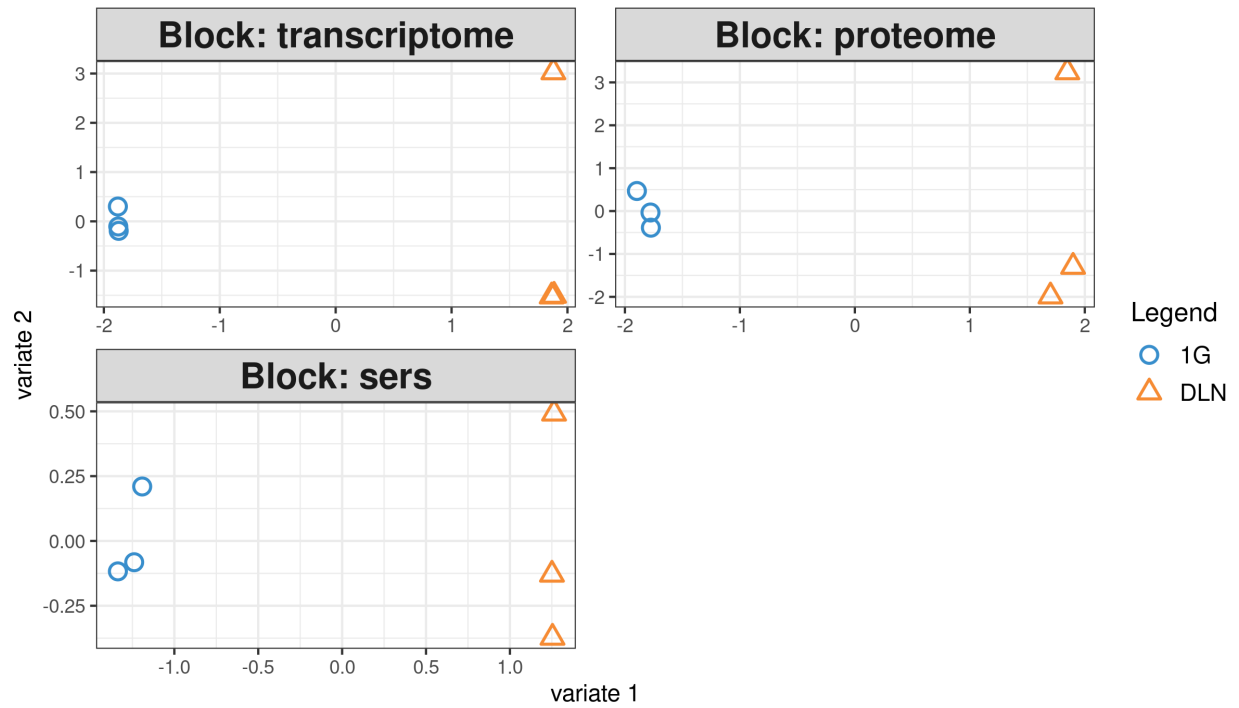
visualization_model <- block.splsda(
  X = data,
  Y = Y,
  ncomp = 2,
  keepX = list(
    transcriptome = c(5, 5),
    proteome = c(5, 5),
    sers = c(2, 2)
  ),
  design = design,
  near.zero.var = TRUE
)
```

```
plotDiablo(
  visualization_model,
  ncomp = 1
)
```

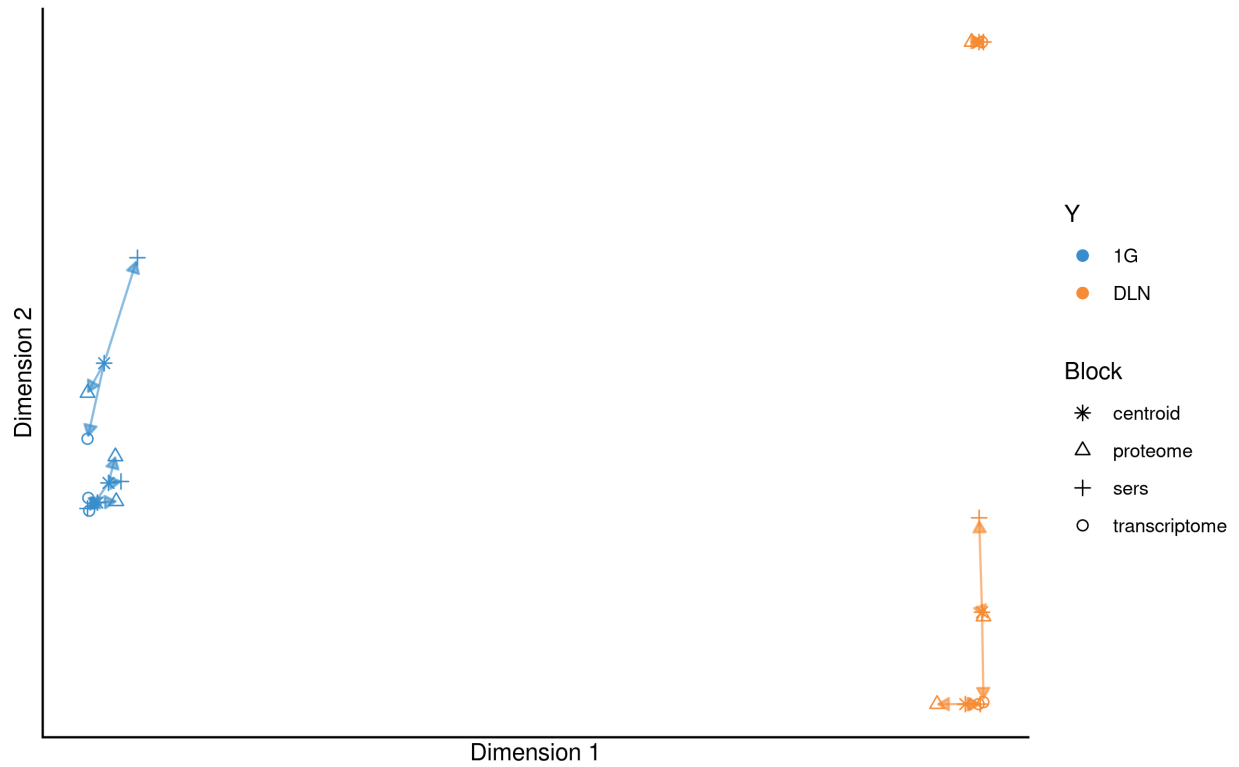


```
plotIndiv(
  visualization_model,
  comp = c(1, 2),
  ind.names = FALSE,
  legend = TRUE,
  title = "DIABLO sample plot"
)
```

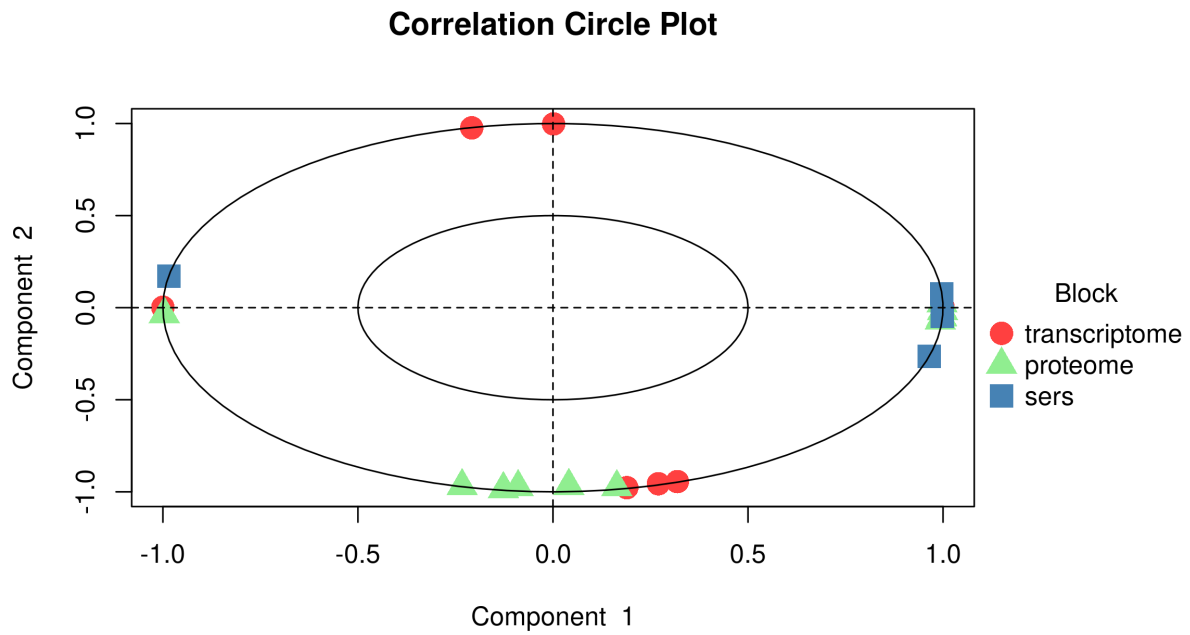
DIABLO sample plot



```
plotArrow(  
  visualization_model,  
  comp = c(1, 2),  
  ind.names = FALSE,  
  legend = TRUE,  
  title = "DIABLO multi-omics sample agreement"  
)
```



```
plotVar(
  visualization_model,
  comp = c(1, 2),
  var.names = FALSE,
  style = "graphics",
  legend = TRUE,
  pch = c(16, 17, 15),
  cex = c(2, 2, 2),
  col = c(
    "brown1",
    "lightgreen",
    "steelblue"
  )
)
```



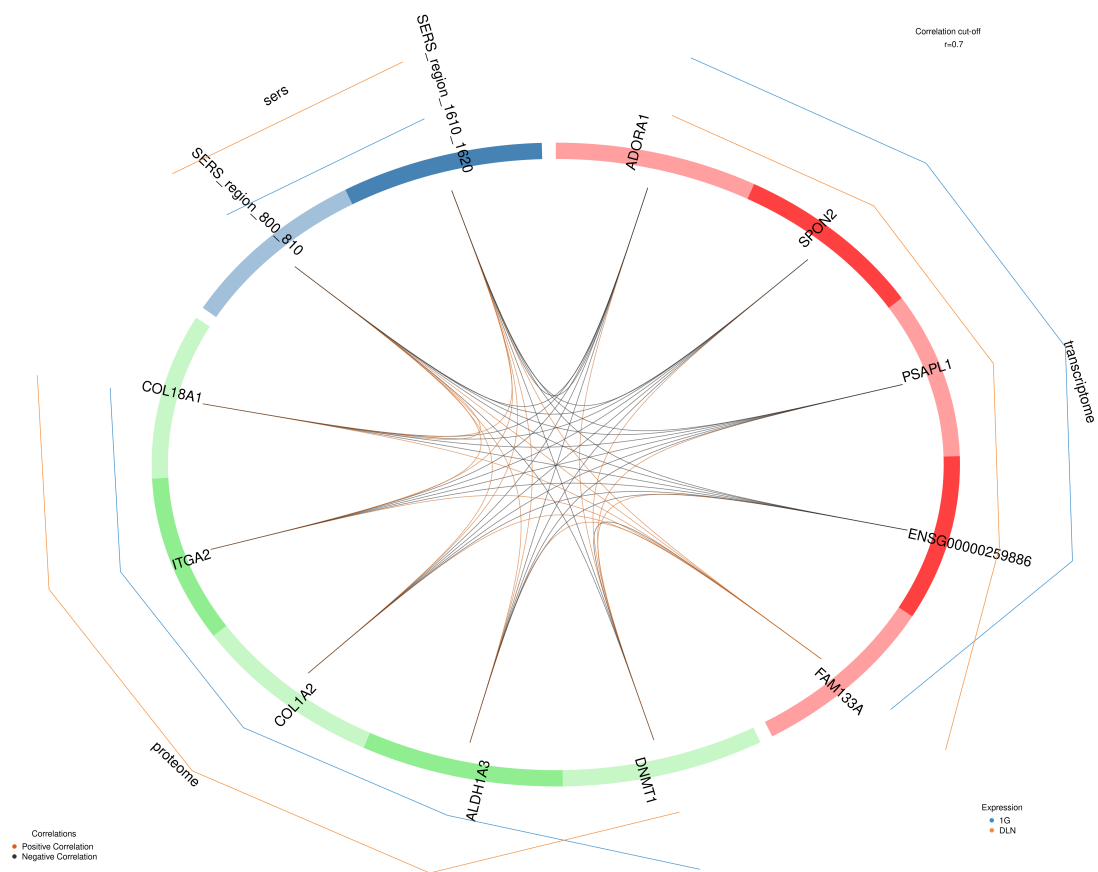
Correlation plots

```

circosPlot(
  final_diablo_model,
  cutoff = 0.7,
  line = TRUE,
  color.blocks = c(
    "brown1",
    "lightgreen",
    "steelblue"
  ),
),
color.cor = c(
  "chocolate3",
  "grey20"
),
size.labels = 1.35,
size.variables = 1.35,
font.labels = 2,
font = 2,
block.line = 3,
lwd = 2
)

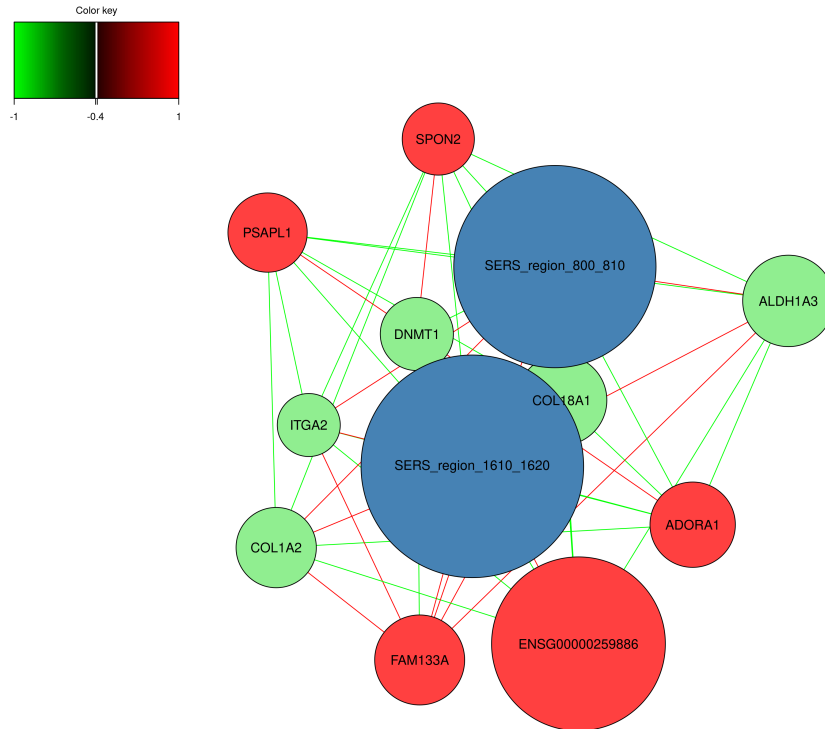
```

ip 1



```
set.seed(123)

mixOmics::network(
  final_diablo_model,
  blocks = c(1, 2, 3),
  color.node = c(
    "brown1",
    "lightgreen",
    "steelblue"
  ),
  cutoff = 0.4,
  size.node = 0.55
)
```



Clustered image map

```
make_labelled_diablo_heatmap <- function(model) {
  block_label_map <- c(
    transcriptome = "Gene",
    proteome = "Protein",
    sers = "SERS"
  )

  selected_matrices <- lapply(names(model$X), function(block_name) {
    block_loadings <- as.matrix(model$loadings[[block_name]])
    selected <- rowSums(abs(block_loadings), na.rm = TRUE) > 0
    model$X[[block_name]][, selected, drop = FALSE]
  })

  names(selected_matrices) <- names(model$X)
  heatmap_matrix <- do.call(cbind, selected_matrices)

  variable_blocks <- unlist(
    Map(
      function(block_name, block_matrix) {
        rep(block_label_map[[block_name]], ncol(block_matrix))
      },
      names(selected_matrices),
      selected_matrices
    )
  )
}
```

```

    ),
    use.names = FALSE
  )

sample_groups <- ifelse(
  as.character(model$Y) == "1G",
  "MCM-1G",
  "MCM-DLN"
)

sample_groups <- factor(
  sample_groups,
  levels = c("MCM-DLN", "MCM-1G")
)

variable_blocks <- factor(
  variable_blocks,
  levels = c("Gene", "Protein", "SERS")
)

sample_order <- order(sample_groups)
heatmap_matrix <- heatmap_matrix[sample_order, , drop = FALSE]
sample_groups <- sample_groups[sample_order]

# Match cimDiablo's display range while retaining the selected values
heatmap_matrix[heatmap_matrix > 2] <- 2
heatmap_matrix[heatmap_matrix < -2] <- -2

# Write the condition names directly into the coloured left blocks
group_annotation <- ComplexHeatmap::rowAnnotation(
  Group = ComplexHeatmap::anno_block(
    gp = grid::gpar(fill = c("#F28E2B", "#2C7FB8"), col = "white"),
    labels = c("MCM-DLN", "MCM-1G"),
    labels_gp = grid::gpar(
      col = "white",
      fontsize = 15,
      fontface = "bold"
    ),
    width = grid::unit(18, "mm")
  ),
  show_annotation_name = FALSE
)

# Write Gene, Protein and SERS directly into the coloured top blocks
block_annotation <- ComplexHeatmap::HeatmapAnnotation(
  Block = ComplexHeatmap::anno_block(
    gp = grid::gpar(
      fill = c("#C44E52", "#55A868", "#4C72B0"),
      col = "white"
    ),
    labels = c("Gene", "Protein", "SERS"),
    labels_gp = grid::gpar(
      col = "white",

```

```

        fontsize = 15,
        fontface = "bold"
    ),
    height = grid::unit(9, "mm")
),
show_annotation_name = FALSE
)

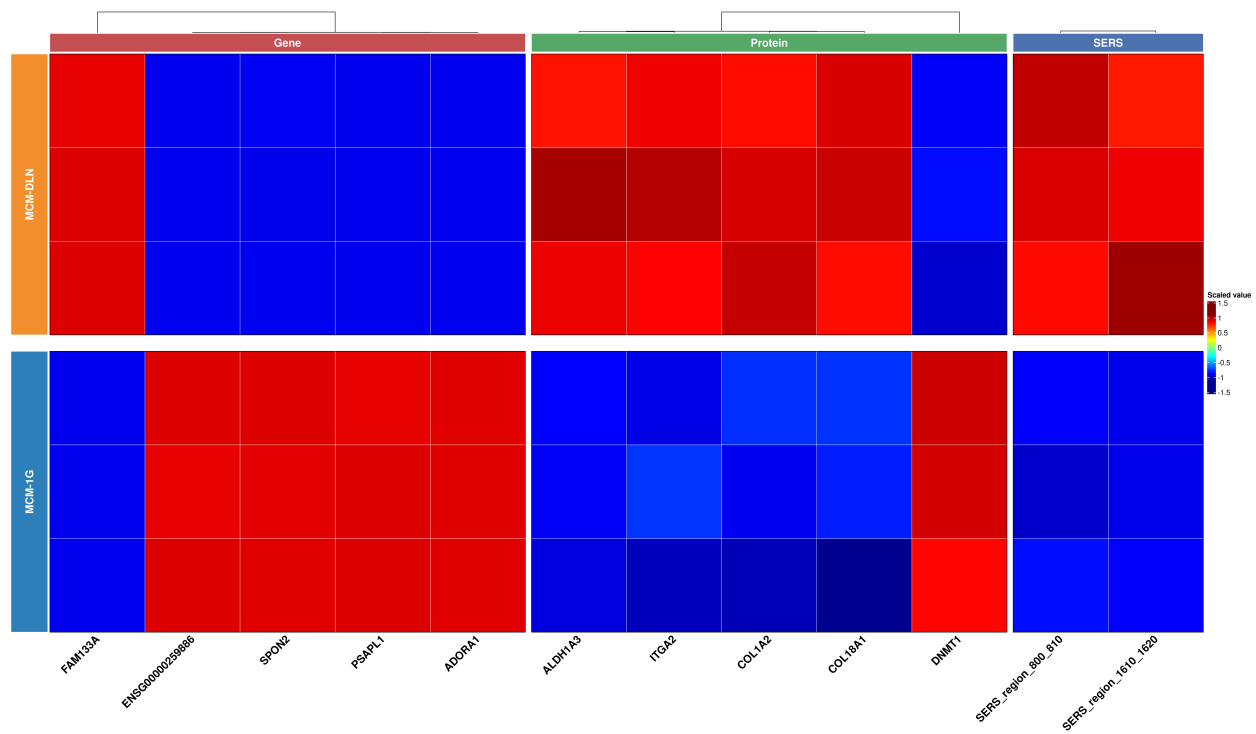
color_limit <- max(abs(heatmap_matrix), na.rm = TRUE)
heatmap_colors <- circlize::colorRamp2(
    seq(-color_limit, color_limit, length.out = 64),
    mixOmics::color.jet(64)
)

ComplexHeatmap::Heatmap(
    heatmap_matrix,
    name = "Scaled value",
    col = heatmap_colors,
    row_split = sample_groups,
    column_split = variable_blocks,
    cluster_rows = FALSE,
    cluster_row_slices = FALSE,
    cluster_columns = TRUE,
    cluster_column_slices = FALSE,
    left_annotation = group_annotation,
    top_annotation = block_annotation,
    show_row_names = FALSE,
    show_column_names = TRUE,
    row_title = NULL,
    column_names_gp = grid::gpar(fontsize = 15, fontface = "bold"),
    column_names_rot = 45,
    column_names_max_height = grid::unit(85, "mm"),
    column_title = NULL,
    row_gap = grid::unit(8, "mm"),
    column_gap = grid::unit(3, "mm"),
    rect_gp = grid::gpar(col = "white", lwd = 0.6),
    border = TRUE,
    heatmap_legend_param = list(
        title = "Scaled value",
        legend_height = grid::unit(45, "mm")
    )
)
}

labelled_diablo_heatmap <- make_labelled_diablo_heatmap(
    final_diablo_model
)

ComplexHeatmap::draw(
    labelled_diablo_heatmap,
    heatmap_legend_side = "right"
)

```



Save figures

```
save_png <- function(
  filename,
  plot_code,
  width = 3000,
  height = 2200,
  resolution = 250
) {
  png(
    filename = file.path(
      figure_dir,
      paste0(sers_method_tag, "_", filename, ".png")
    ),
    width = width,
    height = height,
    res = resolution
  )

  on.exit(dev.off(), add = TRUE)

  eval(
    substitute(plot_code),
    envir = parent.frame()
  )
}
```

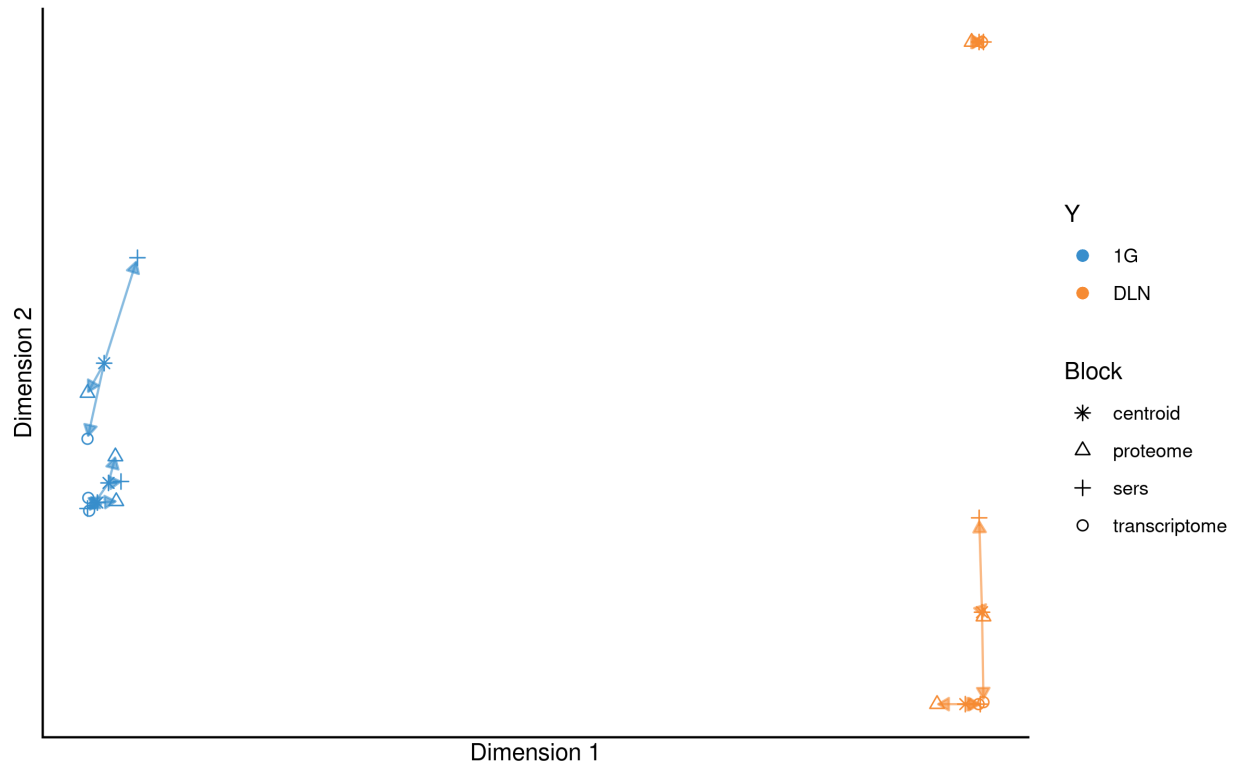
```

save_png(
  "DIABLO_plot",
  plotDiablo(
    visualization_model,
    ncomp = 2
  )
)

save_png(
  "DIABLO_samples",
  plotIndiv(
    visualization_model,
    comp = c(1, 2),
    ind.names = FALSE,
    legend = TRUE,
    title = "DIABLO sample plot"
  )
)

save_png(
  "DIABLO_arrows",
  plotArrow(
    visualization_model,
    comp = c(1, 2),
    ind.names = FALSE,
    legend = TRUE,
    title = "DIABLO multi-omics sample agreement"
  )
)

```



```
save_png(
  "DIABLO_variables",
  plotVar(
    visualization_model,
    comp = c(1, 2),
    var.names = FALSE,
    style = "graphics",
    legend = TRUE,
    pch = c(16, 17, 15),
    cex = c(2, 2, 2),
    col = c(
      "brown1",
      "lightgreen",
      "steelblue"
    )
  )
)
```

```
save_png(
  "DIABLO_circos",
  circosPlot(
    final_diablo_model,
    cutoff = 0.7,
    line = TRUE,
    color.blocks = c(
      "brown1",
      "lightgreen",
      "steelblue"
    )
  )
)
```

```

    ),
    color.cor = c(
      "chocolate3",
      "grey20"
    ),
    size.labels = 1.35,
    size.variables = 1.35,
    font.labels = 2,
    font = 2,
    block.line = 3,
    lwd = 2
  ),
  width = 5000,
  height = 4000,
  resolution = 300
)

save_png(
  "DIABLO_transcriptome_loadings",
  plot_block_loadings(
    final_diablo_model,
    block = "transcriptome",
    comp = 1,
    top_n = 15,
    main = "Transcriptome loadings, component 1"
  )
)

save_png(
  "DIABLO_proteome_loadings",
  plot_block_loadings(
    final_diablo_model,
    block = "proteome",
    comp = 1,
    top_n = 15,
    main = "Proteome loadings, component 1"
  )
)

save_png(
  "DIABLO_SERS_loadings",
  plot_block_loadings(
    final_diablo_model,
    block = "sers",
    comp = 1,
    top_n = 15,
    main = "SERS loadings, component 1"
  )
)

save_png(
  "DIABLO_clustered_image_map",

```

```

ComplexHeatmap::draw(
  labelled_diablo_heatmap,
  heatmap_legend_side = "right"
),
width = 7800,
height = 4800,
resolution = 300
)

cat(
  "Figures saved in:",
  figure_dir,
  "\n"
)

```

```
## Figures saved in: /home/fhwn.ac.at/116914/DIABLO_Melanoma_Pauser/DIABLO_SGL_results/SGL_DIABLO_figur
```

```

network_file <- file.path(
  figure_dir,
  paste0(sers_method_tag, "_DIABLO_network.png")
)

network_candidates <- list.files(
  path = knitr::opts_chunk$get("fig.path"),
  pattern = "^network-plot-[0-9]+[.]png$",
  full.names = TRUE
)

if (length(network_candidates) == 0L) {
  stop("Rendered network figure not found.")
}

network_source <- network_candidates[
  which.max(file.info(network_candidates)$mtime)
]

file.copy(network_source, network_file, overwrite = TRUE)

```

```
## [1] TRUE
```

```
cat("Network saved to:", network_file, "\n")
```

```
## Network saved to: /home/fhwn.ac.at/116914/DIABLO_Melanoma_Pauser/DIABLO_SGL_results/SGL_DIABLO_figur
```

```
sessionInfo()
```

```

## R version 4.5.2 (2025-10-31)
## Platform: x86_64-pc-linux-gnu
## Running under: Ubuntu 20.04.6 LTS
##
## Matrix products: default

```

```

## BLAS: /usr/lib/x86_64-linux-gnu/openblas-pthread/libblas.so.3
## LAPACK: /usr/lib/x86_64-linux-gnu/openblas-pthread/liblapack.so.3; LAPACK version 3.9.0
##
## locale:
## [1] LC_CTYPE=en_US.UTF-8 LC_NUMERIC=C
## [3] LC_TIME=en_US.UTF-8 LC_COLLATE=en_US.UTF-8
## [5] LC_MONETARY=en_US.UTF-8 LC_MESSAGES=en_US.UTF-8
## [7] LC_PAPER=en_US.UTF-8 LC_NAME=C
## [9] LC_ADDRESS=C LC_TELEPHONE=C
## [11] LC_MEASUREMENT=en_US.UTF-8 LC_IDENTIFICATION=C
##
## time zone: Europe/Vienna
## tzcode source: system (glibc)
##
## attached base packages:
## [1] parallel stats4 stats graphics grDevices utils datasets
## [8] methods base
##
## other attached packages:
## [1] GA_3.2.5 msgl_2.3.9
## [3] sglOptim_1.3.8 doParallel_1.0.17
## [5] iterators_1.0.14 foreach_1.5.2
## [7] Matrix_1.7-5 BiocParallel_1.44.0
## [9] org.Hs.eg.db_3.22.0 AnnotationDbi_1.72.0
## [11] igraph_2.3.2 dplyr_1.2.1
## [13] edgeR_4.8.2 limma_3.66.0
## [15] DESeq2_1.50.2 SummarizedExperiment_1.40.0
## [17] Biobase_2.70.0 MatrixGenerics_1.22.0
## [19] matrixStats_1.5.0 GenomicRanges_1.62.1
## [21] Seqinfo_1.0.0 IRanges_2.44.0
## [23] S4Vectors_0.48.1 BiocGenerics_0.56.0
## [25] generics_0.1.4 mixOmics_6.34.0
## [27] ggplot2_4.0.3 lattice_0.22-9
## [29] MASS_7.3-65 knitr_1.51
##
## loaded via a namespace (and not attached):
## [1] DBI_1.3.0 gridExtra_2.3 rlang_1.3.0
## [4] magrittr_2.0.5 clue_0.3-68 GetoptLong_1.1.1
## [7] otel_0.2.0 compiler_4.5.2 RSQLite_3.53.1
## [10] png_0.1-9 vctrs_0.7.3 reshape2_1.4.5
## [13] stringr_1.6.0 pkgconfig_2.0.3 shape_1.4.6.1
## [16] crayon_1.5.3 fastmap_1.2.0 magick_2.9.1
## [19] XVector_0.50.0 labeling_0.4.3 rmarkdown_2.31
## [22] purrr_1.2.2 bit_4.6.0 xfun_0.59
## [25] cachem_1.1.0 blob_1.3.0 DelayedArray_0.36.1
## [28] cluster_2.1.8.3 R6_2.6.1 stringi_1.8.7
## [31] RColorBrewer_1.1-3 Rcpp_1.1.2 tidyselect_1.2.1
## [34] rstudioapi_0.18.0 dichromat_2.0-0.1 abind_1.4-8
## [37] yaml_2.3.12 codetools_0.2-20 tibble_3.3.1
## [40] plyr_1.8.9 withr_3.0.3 rARPACK_0.11-0
## [43] KEGGREST_1.50.0 S7_0.2.2 evaluate_1.0.5
## [46] circlize_0.4.18 Biostrings_2.78.0 pillar_1.11.1
## [49] ellipse_0.5.0 scales_1.4.0 glue_1.8.1
## [52] tools_4.5.2 RSpectra_0.16-2 locfit_1.5-9.12

```

## [55]	grid_4.5.2	Cairo_1.7-0	tidyr_1.3.2
## [58]	colorspace_2.1-2	cli_3.6.6	S4Arrays_1.10.1
## [61]	ComplexHeatmap_2.26.1	corpcor_1.6.10	gtable_0.3.6
## [64]	digest_0.6.39	SparseArray_1.10.10	ggrepel_0.9.8
## [67]	rjson_0.2.23	farver_2.1.2	memoise_2.0.1
## [70]	htmltools_0.5.9	lifecycle_1.0.5	httr_1.4.8
## [73]	GlobalOptions_0.1.4	statmod_1.5.2	bit64_4.8.2